

TITLE OF THE PROJECT

A Major Project Report Submitted in Partial Fulfillment for the Award of the
Degree of Bachelor of Technology in Branch Name

Submitted

To



**Dr. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY,
LUCKNOW**

Submitted By:

NISHANT SINGH BISHT (2200100100228)

NAVYA SINGH (2200100100214)

NISHANT KUMAR SINGH (2200100100225)

SHIVAM SAHANI (2200100100313)

Under the Supervision of

Mr./Dr. Supervisor Name

Assistant Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNITED COLLEGE OF ENGINEERING & RESEARCH,
PRAYAGRAJ, INDIA**

MAY 2026

Certificate

This is to certify that the project titled "**Image Detection**" is the genuine work carried out by **(Student Name)**, **University Roll No. ()**, in partial fulfillment of the requirements for the award of the **Bachelor of Technology** (Computer Science and Engineering) degree submitted to **Dr. A.P.J. Abdul Kalam Technical University, Lucknow**, at **United College of Engineering & Research, Prayagraj**.

The project is an authentic record of their own work carried out during the period from **June 2025** to **May 2026** under the guidance of **Dr./Mr./Ms. (Guide Name)**, **(Designation)**, Department of Computer Science & Engineering.

The Major Project Viva-Voce Exam was held on

Signature of the Supervisor [**Dr./Mr. XYZ**]

Signature of the Project Coordinator [**Mr. Shyam Bahadur Verma**]

Signature of the Head of Department [**Dr. Vijay Kumar Dwivedi**]

Name & Signature of the **External Examiner**

Place: Prayagraj

Date:

Candidate's Declaration

We declare that the project entitled "**Image Detection**", submitted by us in partial fulfillment of the requirements for the award of the degree **Bachelor of Technology**, (Computer Science and Engineering) to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, at United College of Engineering & Research, Prayagraj, is an authentic record of our own work carried out during the period from June 2025 to May 2026 under the guidance of **Dr./Mr./Ms. (Guide Name)**, (Designation of Supervisor), Department of Computer Science and Engineering.

We further declare that the matter presented in this project has not been submitted for the award of any other degree, diploma, fellowship, or other similar title.

Signature of the Student (**XYZ and University Roll No.**)

Signature of the Student (**XYZ and University Roll No.**)

Signature of the Student (**XYZ and University Roll No.**)

Signature of the Student (**XYZ and University Roll No.**)

Place: Prayagraj

Date:

Acknowledgement

We express our sincere gratitude to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, for giving us the opportunity to undertake the Major Project during the final year of our **B.Tech. (Computer Science and Engineering)** program. This project has been an important and enriching learning experience in our engineering education.

We extend our heartfelt thanks to **Dr. Swapnil Srivastava**, Principal, and **Dr. Vijay Kumar Dwivedi**, Dean Academics & Head, Department of Computer Science and Engineering, United College of Engineering & Research, Prayagraj, for their constant encouragement and support.

We owe our deepest gratitude to **Dr./Mr./Ms. (Guide Name)** for his/her valuable guidance, insightful suggestions, and constructive feedback throughout the course of this project, which has been instrumental in its successful completion.

We are also grateful to all those who have directly or indirectly supported us during the project work. Last but not least, we express our appreciation to the authors of the books, research papers, and online resources referred while carrying out the project and preparing this report.

Abstract

Drug discovery is an extremely complex, time-consuming, and resource-intensive activity of which lead identification is a very vital bottleneck during the initial phase. Traditional high-throughput screening systems can only explore a microscopic portion of the estimated 10^{23} – 10^{60} synthetically accessible, drug-like small molecules. In order to overcome this challenge, this project presents a new AI-based drug discovery platform named **SmartChem: An AI-Powered De Novo Drug Discovery Platform** — a generative artificial intelligence framework which enables the exploration of large chemical spaces and the generation of novel, pharmacologically viable compounds.

SmartChem introduces, in place of the weakly-typed Simplified Molecular-Input Line-Entry System (SMILES) representation, a more mathematically robust grammar — **Self-Referencing Embedded Strings (SELFIES)** — which ensures 100% syntactically valid molecular structures, removing post-hoc filtering failures. The architecture consists of a set of **Variational Autoencoders (VAEs)** trained on a curated subset of 100,000 molecules from the ZINC-250k dataset, encoding complex molecular geometries into a smooth, continuous 128-dimensional latent space. Three complementary encoders are employed: a **1D dilated CNN** for sequential SELFIES token dependencies, a **Graph Isomorphism Network (GINEConv)** for molecular topology, and a novel **Hybrid encoder** with a learnable gated modality fusion scheme that dynamically fuses sequence and graph representations.

To prevent the problem of posterior collapse, SmartChem employs a comprehensive anti-collapse training stack including **Cyclical KL Divergence Annealing** (four cycles), a free-bits regularisation floor, 50% selective word dropout, and latent vector concatenation at each GRU decoding step. A **Multi-Layer Perceptron (MLP) Property Predictor** in the latent space is used to predict major pharmacological properties such as QED, LogP, and SAS. Targeted molecular optimisation is performed by gradient descent in the latent space toward user-defined property objectives, and all candidates are subsequently screened through Lipinski

Rule of Five, PAINS, Brenk, and NIH toxicity filters.

The platform is released as a full-stack production web application built with **React**, **TypeScript**, **Vite**, and **WebGL-based 3D visualisation (3Dmol.js)**, supported by an asynchronous **FastAPI** server, **MongoDB** database, and a context-driven AI assistant, **Dr. SmartChem**, powered by the Llama-3.3-70B architecture. Structural validity and novelty were empirically evaluated across 6,750 generation trajectories, confirming **100% structural validity** and **100% novelty** across all generated molecules. The anti-collapse training was successful: the CNN encoder retained 37 active units (28.9% utilisation) and the GNN encoder retained 70 active units (54.7% utilisation). The Hybrid encoder, which integrates both modalities via learned gating, achieved an **18.2% toxicity filter-pass rate** — an **82% relative improvement** over the CNN baseline of 10.0% — and the highest QED targeted-generation hit rate of **7.6%**, demonstrating SmartChem’s efficacy as an end-to-end de novo pharmaceutical lead generation pipeline.

Contents

1	Introduction	1
1.1	Background	1
1.1.1	The Pharmaceutical Drug Discovery Pipeline	2
1.1.2	The Bottleneck of Lead Identification and the Enormity of Chemical Space	3
1.1.3	AI, Machine Learning, and Deep Learning in Drug Discovery	4
1.1.4	Generative Models in Molecular Design	5
1.1.5	Molecular Representations for Machine Learning	6
1.1.6	The SmartChem Approach	7
1.2	Problem Statement	8
1.2.1	Issue 1: The Intractable Vastness and Complexity of Chemical Space	8
1.2.2	Issue 2: Syntactic Brittleness of SMILES and the Invalid Molecule Problem	9
1.2.3	Issue 3: Posterior Collapse in Molecular Variational Autoencoders	9
1.2.4	Issue 4: Fragmented Workflows and the Lack of Integrated Production Platforms	10
1.2.5	Conclusion: SmartChem Motivation	11
1.3	Objectives	12
1.3.1	100% Structural Validity through SELFIES Molecular Representation	12
1.3.2	Training Multi-Modal VAE Encoders and Removing Posterior Collapse	12
1.3.3	Continuous Property Prediction and Dual Latent-Space Optimization	13
1.3.4	Building an Asynchronous Web Platform with Integrated AI Assistance	13

2	Literature Review	15
2.1	Artificial Intelligence in Drug Discovery	15
2.2	Molecular Representation Methods	16
2.3	Variational Autoencoders for Molecule Generation	17
2.4	Graph Neural Networks in Drug Discovery	18
2.5	Hybrid Molecular Learning Models	19
2.6	Posterior Collapse in Variational Autoencoders	19
2.7	Molecular Property Prediction	20
2.8	Existing Drug Discovery Platforms and Research Gap	21
2.9	Summary of Literature Review	21
3	Methodology	23
3.1	Data Collection	23
3.1.1	Dataset Overview	23
3.1.2	Filtering Criteria	24
3.1.3	Preprocessing Pipeline	25
3.2	Algorithms / Models	28
3.2.1	System Architecture Overview	28
3.2.2	SELFIES-Based VAE Pipeline	28
3.2.3	CNN Encoder Architecture	29
3.2.4	GNN Encoder Architecture	30
3.2.5	Hybrid Fusion and Hybrid Encoder	31
3.2.6	Decoder Architecture	32
3.2.7	Anti-Posterior-Collapse Techniques	32
3.2.8	Property Predictor	33
3.2.9	Molecule Generation Modes	34
3.2.10	Toxicity and Drug-Likeness Filters	34
3.3	Full-Stack Platform Architecture	36
3.3.1	Frontend:	36
3.3.2	Backend:	36
3.3.3	Database:	36
3.3.4	Job Queue and Asynchronous Pipeline:	36
3.3.5	Authentication:	37
4	Results and Discussion	38
4.1	Training Performance	39
4.2	Posterior Collapse Analysis	40
4.3	Encoder Architecture Comparison	43
4.4	Property Prediction	44

4.5	Molecule Generation Results	45
4.6	Targeted Generation Results	45
4.7	Toxicity Filtering Results	47
4.8	Platform Performance	49
4.9	Comparison with Related Systems	51
4.10	Strengths and Limitations	51
4.11	Summary	52
5	Conclusion and Future Work	53
5.1	Conclusion	53
5.1.1	Molecular Representation and Generation	53
5.1.2	Encoder Architectures	53
5.1.3	Anti-Posterior-Collapse Training	54
5.1.4	Property Prediction and Targeted Generation	54
5.1.5	Toxicity Filtering and Platform Integration	55
5.1.6	Summary of Results	55
5.2	Future Work	57
5.2.1	Reinforcement Learning Fine-Tuning of the Decoder	57
5.2.2	Correcting Fragment-Size Bias	57
5.2.3	Integration of Molecular Docking and Protein Target Infor- mation	57
5.2.4	Three-Dimensional Geometric Molecular Representations	58
5.2.5	Training on Larger Datasets and for More Epochs	58
5.2.6	Extended Property and Toxicity Endpoint Coverage	58
5.2.7	Wet-Laboratory Validation	59
5.2.8	Cloud Deployment and Scalability	59
5.3	Closing Remarks	60
A	Appendix A	62

List of Figures

4.1	<i>SmartChem molecule generation pipeline. Input molecules are encoded into a continuous latent space, which is either randomly sampled for exploratory generation or gradient-optimised toward user-specified property targets. The GRU decoder produces SELFIES sequences, which are evaluated through a four-stage toxicity filtering pipeline before output.</i>	38
4.2	<i>CNN VAE reconstruction loss over 25 training epochs. Loss decreases from 1.63 to 0.90, indicating stable learning without divergence.</i>	39
4.3	<i>GNN VAE reconstruction loss over 25 training epochs. Loss decreases from 1.81 to 1.10. The higher values relative to CNN reflect the difficulty of the graph-to-sequence cross-modal task.</i>	40
4.4	<i>Active units in the CNN and GNN latent spaces at epoch 25 ($\beta = 1.0$). Dark bars represent active (informative) dimensions; light bars represent collapsed dimensions that have converged to the prior.</i>	41
4.5	<i>Cyclical KL annealing schedule used during training (4 cycles, 25 epochs). β ramps linearly from 0 to 1 in the first half of each cycle, then holds at 1, allowing reconstruction quality to stabilise before regularisation pressure is introduced.</i>	41
4.6	<i>Hybrid encoder architecture. The CNN branch (blue) and GNN branch (green) each produce 64-dimensional projections. A learned sigmoid gate (purple) combines them into the final 128-dimensional hybrid latent vector $\mathbf{z}_{\text{fused}}$.</i>	43
4.7	<i>Representative filter-passing molecules from the CNN (top), GNN (middle), and Hybrid (bottom) encoders. QED and molecular weight (MW) are computed with RDKit. All molecules are chemically valid by SELFIES construction and novel with respect to the training set.</i>	46

4.8	<i>Property hit rates for CNN, GNN, and Hybrid encoders at medium tolerance thresholds. The Hybrid encoder achieves the best QED hit rate (7.6%). Joint property satisfaction (rightmost group) is near zero across all encoders.</i>	47
4.9	<i>Overall filter-pass rates for the CNN, GNN, and Hybrid encoders across 2,250 runs each. The dotted line marks the CNN baseline of 10.0%.</i>	48
4.10	<i>SmartChem web platform architecture. The React frontend communicates with the FastAPI backend, which enqueues generation jobs to MongoDB. An asynchronous ML worker dequeues and processes jobs, then writes results back to MongoDB. The 3Dmol.js viewer and AI assistant operate as browser-side services.</i>	49

List of Tables

4.1	<i>CNN and GNN VAE training performance at epoch 25 ($\beta = 1.0$).</i>	39
4.2	<i>Anti-posterior-collapse techniques used during SmartChem VAE training.</i>	42
4.3	<i>Qualitative comparison of CNN, GNN, and Hybrid encoder architectures.</i>	44
4.4	<i>Molecular properties predicted by the SmartChem multi-output MLP.</i>	44
4.5	<i>Molecule generation quality metrics across all 6,750 evaluation runs.</i>	45
4.6	<i>Targeted generation property hit rates at medium tolerance (QED: ± 0.10, LogP: ± 1.0).</i>	46
4.7	<i>Toxicity and drug-likeness filter-pass rates by encoder (2,250 runs each).</i>	48
4.8	<i>FastAPI backend endpoint mean response times (50-request average).</i>	50
4.9	<i>Mean molecule generation job completion times measured over the async worker pipeline.</i>	50
4.10	<i>SmartChem web platform features and technologies.</i>	50
4.11	<i>Feature comparison of SmartChem with related molecular generation systems.</i>	51
4.12	<i>Strengths and limitations of the SmartChem platform.</i>	52

Chapter 1

Introduction

1.1 Background

Modern pharmaceutical drug discovery is an extraordinarily complex, deeply multifaceted, and historically resource-intensive scientific endeavor. It represents the crucial developmental pipeline through which completely novel chemical entities are discovered, refined, and ultimately approved as life-saving therapeutics. Despite extensive technological advancements in molecular biology, high-throughput chemistry, and automated laboratory robotics over the past century, the timeline for bringing a single new therapeutic drug from its initial conceptualization to formal clinical deployment spans, on average, a grueling **12 to 15 years**, with financial burdens routinely estimated between **1.3 and 2.6 billion USD** per successfully approved medication.

A significant factor contributing to this immense timeline and cost is the profoundly high attrition rate inherently bound to the drug development pipeline. The vast majority of promising molecular candidates ultimately fail during late-stage clinical evaluations due to unforeseen toxicological liabilities, inadequate pharmacokinetic profiles, or insufficient clinical efficacy. While modern rational drug design has shifted the paradigm toward structured campaigns aimed at specific biological mechanisms, the initial stages of molecular discovery remain severely constrained by human intuition and the physical limitations of laboratory screening. Consequently, enhancing the accuracy, breadth, and computational intelligence of the earliest stages of molecular sampling has become one of the premier scientific imperatives of the twenty-first century.

1.1.1 The Pharmaceutical Drug Discovery Pipeline

The standard continuum from a biological hypothesis to a market-ready drug is broadly categorized into the following distinct phases:

1. **Target Identification and Validation:** Researchers isolate the pathological mechanisms responsible for a specific disease phenotype and identify a biological macromolecule—most typically a cellular receptor, catalytic enzyme, or ion channel—whose dysregulation is causally associated with the disease state. The target then undergoes rigorous biological validation (via CRISPR-Cas9 gene knockouts, RNA interference, or pharmacological probing) to confirm that modulating its activity produces the desired therapeutic effect without inducing lethal cellular toxicity.
2. **Hit Discovery:** Researchers initiate the search for preliminary chemical compounds capable of interacting with the validated target via **High-Throughput Screening (HTS)**, wherein extensive physical libraries (1 to 10 million pre-synthesized compounds) are biochemically assayed against the target using automated robotics. Compounds demonstrating statistically significant binding activity or enzymatic inhibition are designated as initial *hits*.
3. **Lead Identification:** Hits are inherently rough starting points; they often bind weakly, interact non-specifically with off-target proteins, or possess disastrous physicochemical properties. Lead Identification involves rigorously filtering and chemically evaluating these hits to uncover a viable *lead compound*—one that displays preliminary biochemical behaviors such as reasonable solubility, baseline stability, and early signs of cellular permeability. This stage is universally considered one of the sharpest bottlenecks in medicinal chemistry and serves as the **primary focal target of the SmartChem generative architecture**.
4. **Lead Optimization:** Medicinal chemists enact iterative, incremental structural modifications to the molecular scaffold to amplify target potency and selectivity while optimizing **ADMET** properties—Absorption, Distribution, Metabolism, Excretion, and Toxicity—ultimately yielding a polished pre-clinical candidate molecule.
5. **Preclinical Testing:** The drug candidate undergoes comprehensive *in vitro* and *in vivo* studies to define toxicological parameters, verify metabolic degra-

duction pathways, and guarantee basic systemic safety before human exposure.

6. **Clinical Trials and Regulatory Approval:** **Phase I** investigates human safety and maximum tolerable dosage in healthy volunteers. **Phase II** assesses preliminary efficacy and side-effect profiling in limited patient populations. **Phase III** expands to large-scale, multi-center, double-blind, randomized controlled trials to prove statistically significant therapeutic benefit. Successful clearance leads to regulatory submission via the FDA or EMA and subsequent market deployment.

1.1.2 The Bottleneck of Lead Identification and the Enormity of Chemical Space

Among the phases above, Lead Identification stands out as a particularly formidable computational and chemical barrier. If a lead compound contains fundamental structural flaws that are difficult to engineer out without destroying binding affinity, it will ultimately fail as a drug candidate. The paradigm of purely screening existing, cataloged compound libraries is rapidly reaching diminishing biochemical returns.

The Concept of Chemical Space

In cheminformatics, **chemical space** represents the theoretical, multidimensional mathematical ensemble encompassing all possible synthetically viable small molecules. Organic molecules are formed through combinatorial assembly of atomic building blocks—carbon (C), nitrogen (N), oxygen (O), sulfur (S), phosphorus (P), and various halogens—bound together adhering to the laws of quantum valence, aromaticity, and stereochemistry.

Restricting the domain to molecules with molecular weight under 500 Daltons per **Lipinski’s Rule of Five**, modern computational consensus estimates the drug-like chemical space at a staggering 10^{23} to 10^{60} possible distinct compounds. For context, the observable universe contains roughly 10^{24} stars, yet humanity’s largest chemical databases—PubChem, ChEMBL, and ZINC—collectively index only 100 to 200 million compounds: a fraction indistinguishably close to zero relative to the full chemical universe.

Limitations of Traditional Lead Discovery

Traditional methods suffer from severe systemic limitations:

- **High-Throughput Screening (HTS) Blindness:** Physical screening restricts the search perimeter to compounds already synthesized and documented, blinding researchers to novel structural scaffolds, unique topological geometries, and unseen pharmacophore arrangements that do not exist in contemporary repositories.
- **Manual Molecule Design Limitations:** The refinement of hits into leads is bottlenecked by human intuition. Medicinal chemists naturally favor familiar chemical reactions and carbon scaffolds, subconsciously preventing exploration of unconventional regions of chemical space. Manual design is also inherently discrete—a chemist can alter a methyl group to an ethyl group, but cannot traverse a continuous mathematical gradient of chemical properties to arrive at a radically different yet functionally equivalent architecture.

1.1.3 AI, Machine Learning, and Deep Learning in Drug Discovery

The limitations of traditional workflows have motivated the integration of **Artificial Intelligence (AI)** and **Machine Learning (ML)** for *de novo* computational drug design. Early computational chemistry relied on **Quantitative Structure-Activity Relationships (QSAR)**, correlating physical descriptors with biological activity, though these models were frequently linear and struggled to generalize beyond their training data.

The advent of **Deep Learning (DL)** precipitated a revolution in this domain. Deep neural networks can operate on raw molecular inputs to automatically deduce the abstract rules governing chemical stability, synthetic accessibility, and target affinity. More importantly, deep learning facilitates a paradigm shift from discrete search to continuous optimization—deep generative AI systems empower researchers to interpolate through the continuous mathematical fabric of a learned chemical space, guiding the probabilistic generation of pharmacologically optimized *de novo* candidate molecules from scratch.

1.1.4 Generative Models in Molecular Design

Generative models explicitly learn the underlying probability distribution of training data, $P(X)$, and can be sampled to generate entirely novel instances. Several generative frameworks have been adapted for cheminformatics:

Variational Autoencoders (VAEs)

A VAE consists of an **encoder** and a **decoder**. The encoder maps a discrete molecular input into a compressed, continuous **latent space**, parameterized by a mean vector μ and variance vector σ^2 . The decoder samples from this distribution to reconstruct the original molecule. During training, the VAE optimizes the **Evidence Lower Bound (ELBO)**, balancing reconstruction accuracy against a **Kullback-Leibler (KL) divergence** regularizer that forces the latent space to conform to a standard Gaussian prior—producing a smooth, continuous space where structurally similar molecules cluster together and gradient-based optimization can discover novel compounds.

Generative Adversarial Networks (GANs)

GANs pit a **Generator** and a **Discriminator** against one another in a zero-sum game. While effective for high-fidelity continuous data, GANs notoriously struggle with discrete sequential molecular data due to the non-differentiability of discrete token sampling, and frequently suffer from **mode collapse**—repeatedly generating a small handful of valid structures rather than exploring broader chemical space.

Reinforcement Learning (RL)

An RL *agent* constructs a molecule step-by-step and receives a **reward** based on its biological or chemical viability, computed by scoring functions evaluating LogP, QED, or docking affinity. Through **Proximal Policy Optimization (PPO)**, the agent learns a policy maximizing expected reward—though meticulously tailored reward functions are essential to prevent exploitation of loopholes that yield chemically nonsensical structures.

Graph Neural Networks (GNNs)

Molecules are fundamentally topological graphs—atoms as nodes, bonds as edges. GNNs employ **message passing** mechanisms wherein each atom updates its hidden state by aggregating information from bonded neighbors, comprehensively understanding both local functional groups and overarching structural topologies.

Generative GNN implementations build graphs node-by-node and edge-by-edge, providing detailed structural control over the generative process.

1.1.5 Molecular Representations for Machine Learning

The effectiveness of any deep generative architecture depends critically on the numerical molecular representation chosen.

Molecular Fingerprints

Fingerprints such as **ECFP** (Morgan fingerprints) represent molecules as fixed-length binary vectors by hashing local atomic neighborhoods. While efficient for similarity mapping and QSAR modeling, fingerprints are a **lossy representation**—hashing collisions erase structural specifics, making inversion back to a distinct molecular graph mathematically impossible. They are therefore unsuitable as output targets for deep generative decoders.

The SMILES Notation and Its Generative Limitations

SMILES encodes molecular topologies into compact ASCII strings (e.g., Benzene as `c1ccccc1`) and served as the *de facto* standard for molecular data exchange. However, SMILES possesses a **catastrophic failure mode** in generative contexts—a character-by-character decoder can easily misplace a parenthesis, omit a ring closure digit, or create impossible atom valences. Foundational SMILES-based VAE studies verified that only **35% to 65%** of decoded strings resolved into valid molecules, meaning over half of all generated hypotheses had to be discarded post-generation.

SELFIES: Robust Strings by Design

SELFIES (Self-Referencing Embedded Strings) utilizes a Chomsky-like grammar that inherently enforces quantum valence and structural rules during string assembly. **Every possible combinatorial string over the SELFIES alphabet corresponds definitively to a syntactically valid molecular graph**—if a decoder generates an impossible bond, the SELFIES parser automatically resolves it into a valid structural equivalent. Post-generation validity rates therefore natively approach **100%**, making SELFIES the foundational sequential language driving SmartChem.

Graph Representations

Molecular graphs encode molecules as undirected graphs maintaining full chemical topology without linearization artifacts, defined by:

- **Node Features (Atoms):** Atomic number, connectivity degree, formal charge, hybridization state, and aromaticity.
- **Edge Features (Bonds):** Bond type (single, double, triple, aromatic) and 3D stereochemical geometry.

Feeding these graph structures directly into GNNs allows deep learning models to natively dissect the topological complexities of heterocycles and branching chains without artificial sequence constraints.

1.1.6 The SmartChem Approach

SmartChem: An AI-Powered De Novo Drug Discovery Platform is engineered to address the deep-rooted problems of pharmaceutical Lead Identification. It wholly abandons SMILES in favor of **SELFIES**, and implements a **tripartite VAE architecture** comprising a **1D CNN** encoder for sequential SELFIES structures, a **GNN** encoder for raw molecular graphs, and a novel **Hybrid Encoder** with a gated modality fusion mechanism that dynamically weighs and unifies both modalities via parametric gating layers.

To prevent **posterior collapse**, SmartChem deploys **cyclical KL annealing** and per-dimension **free-bits** algorithms. An integrated **MLP** property predictor enables rigorous gradient-based and Bayesian latent space optimization toward pharmacological parameters such as QED and LogP. The platform is delivered as a production-grade full-stack web application built with **React**, **TypeScript**, and **FastAPI**, with **MongoDB** handling both data orchestration and atomic asynchronous job queueing—offering continuous, secure, AI-assisted therapeutic candidate generation within an intuitive pharmaceutical interface.

1.2 Problem Statement

Artificial intelligence and deep generative modeling hold tremendous promise for accelerating the Lead Identification step of the therapeutic development pipeline. Nonetheless, even with rapid theoretical progress in cheminformatics and deep neural architectures, translating these theoretical models into viable, production-quality drug discovery engines remains a profound challenge. Even the most advanced computational systems are severely constrained by mathematical, representational, and infrastructural limitations that restrict their practical utility. This project specifically outlines and tackles four critical technical and systemic issues with modern AI-based molecular design that together drive the architecture of the SmartChem platform.

1.2.1 Issue 1: The Intractable Vastness and Complexity of Chemical Space

The most fundamental mathematical challenge in computational drug discovery is the astronomical, non-computable size of chemical space. The number of chemically stable, drug-like molecules (below 500 Daltons) is estimated to lie between 10^{23} and 10^{60} , derived purely from the combinatorics of atomic building blocks—carbon, nitrogen, oxygen, sulfur, and the halogens—constrained by the physical rules of quantum valence and spatiotemporal stability.

By comparison, the largest digital compound collections ever assembled by humanity, including the ZINC and PubChem databases, index only tens to hundreds of millions of distinct synthesized entries. Conventional High-Throughput Screening (HTS) and manual design workflows are therefore fundamentally trapped, restricted to searching a negligible subset of known scaffolds. Exhaustive or discrete algorithmic search over this domain is mathematically non-computable. To be successful, AI generative architectures must not only navigate an environment mathematically analogous to searching the entire observable universe for one functional star, but must also extrapolate well beyond their training distributions without collapsing into the production of random, pharmacologically unviable chemical noise. Efficient navigation of this vast multidimensional space toward pharmacologically relevant targets remains an outstanding unsolved problem in contemporary *de novo* design.

1.2.2 Issue 2: Syntactic Brittleness of SMILES and the Invalid Molecule Problem

Complex organic molecules must be numerically serialized for processing by artificial neural networks. The **Simplified Molecular-Input Line-Entry System (SMILES)** has historically been the dominant encoding standard in the deep learning community, representing molecular structures—cyclizations, aliphatic side chains, and stereochemistry—as compact, 1D context-free strings. However, SMILES carries a disastrous system-wide weakness when coupled with probabilistic sequence generation: its extreme syntactic fragility.

When a generative deep learning agent decodes a new molecular structure autoregressively, character by character, a single erroneous token prediction causes complete catastrophic breakdown. Predicting an unclosed branch parenthesis, instantiating an unmatched ring-closure digit, or assigning more bonds to an atom than quantum valence permits renders the entire string computationally invalid.

In foundational research assessing deep generative molecular models, it was consistently found that **35% to 65%** of SMILES strings generated from continuous latent variables were syntactically invalid. This is a serious architectural flaw. Generative networks waste enormous computational bandwidth learning to navigate artificial string formatting rules rather than generalizing the physicochemical logic of molecular space. Approximately half of all molecular hypotheses produced by classical AI platforms must be discarded post-generation. This further introduces a strong biasing effect on optimization trajectories and latent space continuity, where small gradient steps can push the model into regions that produce nothing but nonsensical ASCII sequences. Despite these deep-seated weaknesses, the overwhelming majority of legacy academic and commercial systems remain bound to the SMILES representation protocol, crippling generative efficiency.

1.2.3 Issue 3: Posterior Collapse in Molecular Variational Autoencoders

Variational Autoencoders (VAEs) are potent generative engines trained to encode discrete molecules into a continuous, lower-dimensional latent space, enabling gradient-based optimization of molecular properties. However, VAEs are susceptible to a debilitating failure mode when operating on discrete sequential data such as molecular string notation, known in the deep learning literature as **posterior collapse**.

The collapse arises from the optimization dynamics between the VAE encoder and its powerful autoregressive sequence decoder (typically GRU or LSTM networks). Because the decoder can reconstruct the next token using only localized token memory and immediately preceding context, the mathematical optimization landscape is strongly biased toward a degenerate solution during training: the decoder learns to perform perfect step-by-step molecular reconstruction while systematically disregarding the latent vector signal z supplied by the encoder. When this occurs, the approximate posterior $q(z|x)$ mathematically collapses onto the uninformative standard Gaussian prior $p(z)$.

In this collapsed state, the latent vector carries zero meaningful molecular, geometric, or structural information. Poorly regularized molecular VAEs regularly report deplorable Active Unit counts—often only one tenth to one fifth of the full latent dimensionality remaining functional. Random sampling from these collapsed regions blindly generates structurally barren, repetitive outputs. More critically, gradient-based latent space interpolation and structural optimization become mathematically undefined, as entirely distinct molecular structures map to the same uninformative coordinates in the dead subspace, completely preventing smooth traversal of chemical space.

1.2.4 Issue 4: Fragmented Workflows and the Lack of Integrated Production Platforms

The final monumental impediment to the widespread clinical adoption of AI-driven *de novo* molecular generation is infrastructural. Although highly sophisticated algorithmic models exist, the overwhelming majority of machine learning molecular frameworks exist only as fragmented, unoptimized batch-processing Python scripts locked within ad-hoc command-line environments.

A medicinal chemist attempting to identify a new lead candidate is forced to context-switch across deeply disconnected software environments: running standalone multi-GPU command-line scripts to generate VAE molecular candidates, manually exporting files through custom RDKit pipelines for ADMET filtering and Lipinski compliance checks, uploading passing molecules to discrete cheminformatics web servers for 3D structural visualization, and consulting external large language models for chemical interpretation—all as entirely separate, manually coordinated operations. No single unified application natively and synchronously covers these fundamentally interdependent tasks.

A desperate, unmet need exists for an integrated, intuitive, high-concurrency, production-grade platform that abstractly encapsulates complex generative mechanics, safely handles parallel computational loads, autonomously executes Lipinski and toxicity filtering (PAINS, Brenk, NIH), and visualizes optimized molecular profiles within an accessible graphical interface—without requiring deep command-line expertise from the end user.

1.2.5 Conclusion: SmartChem Motivation

The astronomically unnavigable combinatorics of chemical space, the catastrophic invalidity rates of the brittle SMILES encoding language, the debilitating loss of latent utility due to posterior collapse, and the deeply fragmented nature of practical pharmacological software are four specific, systemic challenges that unavoidably stall the rapid practical implementation of AI-based molecular generative design. To produce a scalable, practically implementable computational mechanism to overcome this territory, it is absolutely necessary to replace legacy syntax with mathematically guaranteed string systems, to impose strict anti-collapse regularizers to restore continuous VAE latent spaces, and to package the full analytical engine within a seamlessly integrated, hardware-accelerated software environment. It is precisely this need to overcome these four overt technical constraints that drives the multidisciplinary research, architectural design, and end-to-end full-stack software development of the SmartChem platform.

1.3 Objectives

The main goal of this project is to establish an end-to-end artificial intelligence platform that can be practically used to fast-track the lead identification stage in computational drug discovery. In order to achieve the SmartChem architecture successfully, the project concentrates on four key interrelated goals:

1.3.1 100% Structural Validity through SELFIES Molecular Representation

Most of the current deep learning models cannot produce valid chemicals due to the use of brittle SMILES strings. Generative models predicting character by character in SMILES often make syntax errors leading to broken, syntactically inept molecules. The initial task we aim to complete in order to break this basic bottleneck is the implementation of the SELFIES (Self-Referencing Embedded Strings) representation format. SELFIES has an extremely strong grammatical backbone, which imposes valence and ring-closure constraints in a dynamic fashion. The model cannot produce a chemically invalid structure due to this inbuilt mathematical safety net. Our generative models will naturally generate *de novo* molecules with **100% structural validity** in their native form by tokenizing and processing SELFIES. This makes sure that computational bandwidth is ruthlessly used to identify promising drug candidates instead of again and again eliminating algorithmic gibberish.

1.3.2 Training Multi-Modal VAE Encoders and Removing Posterior Collapse

Due to the presence of sequential text features and 2D topological graph features in chemical structures, we want to train a tripartite **Variational Autoencoder (VAE)** architecture. The aim of this is to construct discrete CNN and GNN encoders, as well as a new Hybrid encoder that has been designed to dynamically combine the sequential knowledge of the CNN with the topological geometry of the GNN with the help of a parametric gating process.

Another critical problem with training such sequential VAEs is the mode of failure known as **posterior collapse**: a drastic failure mode where the decoder fully learns to ignore the compressed latent space. We shall forcefully solve this

by putting into practice an aggressive anti-collapse training program, which is directly aimed at latent reactivation. This strict training protocol is solidly anchored with the use of **cyclical KL divergence annealing** to reset the dependency measures at each training step, periodic regularization of **free bits** to avoid dimensional collapse, and systematic dropout of decoder words. These particular methods mathematically ensure that the continuous latent space is an effective way of representing rich, actionable chemical information.

1.3.3 Continuous Property Prediction and Dual Latent-Space Optimization

After successfully creating a structurally sound, non-collapsed latent space, the goal switches to actionable drug design. An optimized **Multi-Layer Perceptron (MLP)** neural network will be attached to the latent embeddings and used as a precise biomolecular property predictor, specifically trained to estimate essential drug-like properties with a primary emphasis on the **Quantitative Estimate of Drug-likeness (QED)**, compound lipophilicity (**LogP**), and the estimated **Synthetic Accessibility Score (SAS)**.

By fully differentiating this predictive space, two very specific mathematical optimization problems are made solvable. We will explicitly use **Adam-based gradient ascent** methods with L2 anchoring constraints to safely run high-speed, local optimizations given a seed molecule. At the same time, we will perform sweeping **Bayesian Optimization** protocols on a compressed PCA subspace to perform a global search and discover entirely new molecular scaffolds that satisfy stringent biological properties.

1.3.4 Building an Asynchronous Web Platform with Integrated AI Assistance

To properly implement our deep learning scripts as a useful software product for medicinal researchers, the last goal is to create a highly scalable, full-stack web application. The frontend will be implemented using **React** and **TypeScript**, with explicit use of **3Dmol.js** to provide high-fidelity, real-time 3D molecule conformations in the browser. The backend will be based on an object-oriented **Python FastAPI** framework to support parallel computational workflows securely.

To avoid stalling the heavy PyTorch local inference engine with regular HTTP requests, the platform will uniquely configure **MongoDB** to serve simultaneously as both a native asynchronous background machine learning Task Queue and a synchronous data store. Moreover, any AI-produced molecular hypothesis will be automatically subjected to strict **RDKit** toxicity filtering constraints, automatically eliminating structures that violate Lipinski rules or trigger **PAINS** or **Brenk** catalog warnings. Lastly, an embedded conversational AI agent driven by the **Groq Llama-3.3** analytical model will be deployed in-situ to give users hallucination-free, context-sensitive biological information about their explicitly generated compounds.

Chapter 2

Literature Review

2.1 Artificial Intelligence in Drug Discovery

Artificial Intelligence (AI) has become one of the most disruptive technologies in the field of pharmaceutical research. The conventional drug discovery process is a very costly, lengthy and risky venture. The average time and expense of creating a new drug is estimated to be over 10 years and billions of dollars, whereas most candidate drugs fail in clinical trials. These constraints have prompted the use of AI-based solutions that can help speed up the decision-making process, minimize expenses, and enhance the drug development success rate.

AI has been applied in nearly all phases of the drug discovery pipeline. Machine learning models can be used in the process of identifying the target, analyzing biological data, including genomics, transcriptomics, and proteomics, to determine proteins or genes related to a disease. In the process of identifying leads, the AI system can computationally screen millions of compounds to find molecules that are likely to bind to a specific target protein. In the process of lead optimization, AI assists the researchers to optimize existing molecules to enhance their properties, including potency, toxicity, stability, and solubility.

Molecular generation and property prediction have become particularly significant areas of deep learning models. The modern drug discovery systems increasingly make use of Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), Graph Neural Networks (GNNs), reinforcement learning methods, and transformer-based language models. Such approaches are capable of acquiring complicated relationships between molecular structure and biological activity, allowing the production of novel drug-like molecules.

Toxicity prediction and ADMET evaluation have also been enhanced with the use of AI-based systems. With large datasets like Tox21 and ClinTox, machine learning can be used to predict the likelihood of a molecule causing liver toxicity, cardiotoxicity, mutagenicity, or poor absorption. This will assist in filtering out inappropriate candidates at earlier stages of the pipeline and limits chances of failure at later stages.

Although these advantages are present, there are notable limitations of current AI systems. Most models need extremely large datasets to train on, and the quality of available data is usually unexamined. Deep learning models are also not easy to interpret, and as such, researchers cannot tell why a certain prediction was made. Moreover, a variety of systems continue to have difficulties in achieving the synthesis of various types of molecular data including sequence information, graph structure, and biological context. Such restrictions encourage the creation of stronger and more integrated platforms like SmartChem.

2.2 Molecular Representation Methods

One of the most significant design options in computational drug discovery is molecular representation, since the performance of a machine learning model is directly related to the encoding of molecules. SMILES strings, SELFIES strings, molecular fingerprints, and graph-based structures are the most ubiquitous molecular representations.

The most popular molecular representation is called **SMILES** (Simplified Molecular Input Line Entry System). It stores the structure of a molecule as a text string. SMILES is readable, standardized, and compatible with cheminformatics libraries including RDKit. However, it has a major drawback in generative modeling. Any minor modification in a SMILES string tends to give invalid molecules due to the weak syntax. One lost bracket, wrong bond symbol, or disturbed token can invalidate the whole molecule.

In order to resolve this issue, **SELFIES** (Self-Referencing Embedded Strings) was introduced. SELFIES is a robust molecular string encoding whereby every possible string represents a chemically valid molecule. This renders SELFIES the best tool for molecular generation activities as it entirely removes invalid outputs brought about by syntax errors. Although SELFIES is not as human-readable as SMILES, its consistency renders it more practical in AI-driven molecular genera-

tion.

Molecular fingerprints are fixed-length molecular vectors. Major methods such as Extended Connectivity Fingerprints (ECFPs) record the existence of certain molecular substructures. Fingerprints are computationally inexpensive and are frequently utilized in similarity search and QSAR modeling. They however lose valuable structural information and are unable to rebuild the original molecule.

Graph representations represent molecules as graphs, with atoms as nodes and bonds as edges. This description is intuitive as molecules are graph structures in nature. Graph-based representations preserve topological information and are ideal for Graph Neural Networks. They also have permutation invariance, meaning the order of atoms does not affect the representation. Nonetheless, graph-based generation is more complex and computationally intensive compared to string-based approaches.

Each representation carries both strengths and weaknesses. SMILES is simple to operate yet prone to invalidity. SELFIES guarantees validity and suits generative models. Fingerprints are effective in prediction tasks but lose structural detail. Graphs are the most informationally rich and are fundamental to learning via GNNs. SmartChem combines SELFIES and graph representations to take advantage of both methods.

2.3 Variational Autoencoders for Molecule Generation

Among generative models for computational chemistry, **Variational Autoencoders (VAEs)** are particularly popular. A VAE contains two components: an encoder which maps a molecule into a continuous latent representation, and a decoder that reconstructs a molecule given a latent vector. Unlike a standard autoencoder which produces a single deterministic latent point, VAEs optimize a probabilistic latent space which makes interpolation and optimization more stable.

The usefulness of VAEs for molecular generation was first demonstrated by Gómez-Bombarelli et al., who showed that it was possible to embed molecules into a continuous latent space and optimize a molecule’s latent vector toward a desired property. The key idea was to navigate chemical space through latent

vectors rather than by modifying molecules directly.

An important benefit of VAEs is that the latent vector can be optimized in a gradient-based manner to produce molecules with desirable features. A molecule can be encoded to a latent vector, and this vector can subsequently be modified to increase properties such as drug-likeness and solubility while decreasing toxicity. However, VAEs also come with critical limitations. The main problem with SMILES-based VAEs is the high occurrence of invalid molecular structures. Another significant issue is **posterior collapse**, where the decoder learns to reconstruct molecules independently of the latent vector, rendering the latent space useless and making any latent space optimization efforts futile.

SELFIES provides a better molecular representation than SMILES as it is unable to produce invalid molecules. Many modern VAE architectures also incorporate methods such as KL annealing, free bits, word dropout, and latent injection in order to tackle posterior collapse, leading to highly versatile models for molecular generation.

2.4 Graph Neural Networks in Drug Discovery

Graph Neural Networks (GNNs) are a fundamental tool in drug discovery due to the natural mapping between molecules and graphs, where atoms represent nodes and bonds represent edges. Atoms can be assigned specific features such as atomic type or hybridization, whereas bond features can represent bond type or aromaticity.

Message passing is the core mechanism of GNNs. Each node aggregates information from neighboring nodes to update its representation. With increasing layers, information from further regions of the molecule is incorporated, which is extremely useful for molecular property prediction and representation learning.

Popular GNN variants include **Graph Convolutional Networks (GCNs)**, which aggregate neighbors with weighted averages, and **Graph Attention Networks (GATs)**, which learn attention weights during aggregation. **Message Passing Neural Networks (MPNNs)** were designed to explicitly include bond features in the message function, allowing richer feature interactions. Expressive GNNs such as the **Graph Isomorphism Network (GIN)** and **Graph Isomorphism Network with Edge Features (GINEConv)** utilize sum aggregation,

making them better than average-aggregation networks at discriminating between graph structures. These have become some of the most powerful GNNs for molecular property prediction tasks such as predicting toxicity, solubility, and bioactivity. GNNs can also be used as the encoder within a VAE, though overuse of layers can result in oversmoothing and indistinguishable node representations.

2.5 Hybrid Molecular Learning Models

Hybrid molecular learning models combine several molecular representations to learn a more complete model of chemical structure. They are founded on the principle that each representation captures different aspects of a molecule: sequential representations such as SELFIES capture long-range sequence trends, whereas graph representations capture local topological interactions. Separate encoders are used for each representation, and the extracted features are subsequently combined by fusion.

The advantages of hybrid models stem from the complementary information that each representation provides. Graphs capture neighborhood interactions while sequences capture more long-range phenomena. Hybrid models often perform better on tasks like property prediction and generation than unimodal models. The downside is their increased complexity; multi-modal models are more difficult to optimize and require longer training times due to the numerous encoders involved. The SmartChem system employs a hybrid architecture combining a CNN for sequence modeling and a GNN for graph modeling, enabling more feature-rich latent representations.

2.6 Posterior Collapse in Variational Autoencoders

Posterior collapse is a crucial challenge when training Variational Autoencoders. It results in the encoder representing the data’s posterior distribution as extremely similar to the prior distribution, making the latent space almost useless for storing information. In turn, the decoder entirely relies on its own internal structure rather than conditioning on the latent vector. Posterior collapse tends to occur most frequently when a powerful decoder such as an RNN or transformer is employed, as these are capable of reconstructing sequences solely through their internal components, rendering the KL divergence loss minimal.

In molecular generation this is especially problematic, as the latent space is relied upon for interpolation, property-driven searching, and generation. Several popular techniques are used to reduce the impact of posterior collapse. **KL annealing** involves gradually increasing the weighting of the KL divergence term, allowing the model to concentrate on reconstruction before imposing heavy regularization; **cyclical KL annealing** then applies repeated cycles of this process. **Free bits** imposes a floor on the KL divergence to force each latent dimension to carry a minimal amount of information. **Word dropout** on sequences is another widely used technique. Architectural changes such as simplifying the decoder or injecting the latent vector at every decoding step (latent injection) have also proven effective. The SmartChem model utilizes multiple anti-collapse methods simultaneously, including cyclical KL annealing, free bits, word dropout, latent injection, and a constrained decoder design.

2.7 Molecular Property Prediction

Molecular property prediction is an integral component of computational chemistry, serving as a way to screen potential candidates by identifying molecules which possess the desired qualities for drug discovery. Instead of experimentally testing each molecule, machine learning models can predict specific properties directly from molecular structure.

Key properties of interest include **QED** (Quantitative Estimate of Drug-likeness, which measures how drug-like a molecule is compared to known oral drugs), **LogP** (the balance between hydrophilic and lipophilic portions of a molecule, correlating to membrane permeability), and **SAS** (Synthetic Accessibility Score, estimating how easy or difficult a molecule would be to synthesize in practice), in addition to toxicity and general drug-likeness metrics.

Molecular fingerprints such as ECFP vectors have been used efficiently for property prediction, though they tend to lose structural information useful for optimization. Modern deep learning models such as GNNs now learn directly from molecular graphs to increase prediction accuracy. Properties can also be predicted from the latent vector derived from a VAE via a separate neural network, which additionally permits manipulation of the latent vector in directions that yield favorable properties. Popular benchmarking datasets include MoleculeNet, Tox21, and ClinTox. Property prediction is central to SmartChem, where QED, LogP, and SAS are predicted directly from molecular latent vectors, providing ranking,

scoring, and optimization capabilities based on drug-likeness.

2.8 Existing Drug Discovery Platforms and Research Gap

Various AI platforms aimed at facilitating drug discovery have been introduced over the years. **REINVENT** uses Reinforcement Learning and SMILES strings for generation and reward-based optimization. **MolGAN** and **JT-VAE** were among the first to explore graph-based methods and the concept of generating molecules as valid sub-structures respectively.

However, these platforms all suffer from a range of shortcomings. Most generate SMILES strings, making invalid molecule generation commonplace and computationally wasteful. Many exist only as command-line tools and do not incorporate features such as 3D viewers or interactive modification functionalities, hindering practical use. Most VAE-based approaches also suffer from posterior collapse, losing important latent information necessary for proper generation and optimization, and typically rely on only a single molecular representation.

SmartChem was developed to tackle these issues by using SELFIES to guarantee the validity of generated molecules, employing hybrid CNN and GNN encoders, incorporating multiple techniques to prevent posterior collapse, integrating a toxicity filter and molecular property predictor, and delivering all functionality within an interactive 3D web platform. The literature thus confirms that while existing methods have demonstrated utility, further development of effective, interactive, and comprehensive AI tools remains necessary for optimal drug discovery.

2.9 Summary of Literature Review

The literature shows that AI has become an essential tool in modern drug discovery. Machine learning and deep learning models are now widely used for molecular generation, property prediction, toxicity analysis, and lead optimization. Among the various molecular representations, SELFIES provides the highest robustness for generation because it guarantees valid outputs, while graph-based representations are best suited for capturing molecular topology.

Variational Autoencoders remain one of the most useful frameworks for learning continuous molecular latent spaces. However, they are highly vulnerable to posterior collapse, which reduces the usefulness of latent representations. Various methods such as KL annealing, free bits, and word dropout have been proposed to address this problem.

Graph Neural Networks have demonstrated strong performance in molecular property prediction because they directly model atoms and bonds as graph structures. Recent research has also shown that hybrid models combining sequence-based and graph-based representations can outperform single-modality approaches.

Although many existing platforms provide useful tools for molecular generation, they often suffer from problems such as invalid molecule generation, lack of multimodal representations, weak visualization support, and poor latent space quality. These limitations motivate the development of SmartChem, which combines SELFIES, hybrid CNN-GNN encoders, anti-collapse training strategies, molecular property prediction, toxicity filtering, and an integrated web platform into a single unified system.

Chapter 3

Methodology

3.1 Data Collection

3.1.1 Dataset Overview

The quality, scale, and diversity of the chemical dataset used to train any machine learning-based generative model to design molecules are fundamental to its development. In the case of the SmartChem platform, the ZINC-250k dataset was chosen as the main source of molecular data. ZINC-250k is a well-known standard dataset in computational drug discovery, comprising 250,000 commercially available, drug-like organic molecules selected from the larger ZINC database — a free database of over one billion purchasable chemical compounds curated by the Irwin and Shoichet Laboratories at the University of California, San Francisco.

ZINC-250k has been used extensively in previous molecular generation literature, including graph-based generative models, variational autoencoders, and reinforcement learning-based optimization methods. Its widespread usage in the literature makes it a suitable and defensible choice for training and evaluating a system such as SmartChem, which is aimed at creating novel, chemically valid, and drug-like molecules. Every molecule within the dataset is encoded in the SMILES (Simplified Molecular Input Line Entry System) format — a succinct string-based representation of molecular structure that is human-readable and broadly compatible with cheminformatics toolkits, including RDKit.

For this project, 100,000 molecules were selected from the full ZINC-250k dataset for training and evaluation. This subset was chosen using a set of filtering and validation criteria to maintain data quality and consistency within the constraints imposed by the model architecture.

3.1.2 Filtering Criteria

Before molecules were added to the training pipeline, every candidate molecule in the ZINC-250k dataset was filtered through a series of validation protocols. The aim was to retain only those molecules meeting the structural and representational criteria required for reliable model training. Molecules failing any of the following criteria were excluded:

1. **Maximum Heavy Atom Count:** Each molecule was required to contain no more than 60 heavy atoms (non-hydrogen atoms). This upper bound was imposed to ensure that graph representations of molecules could be computed with reasonable computational effort and to prevent overly large inputs to the Graph Neural Network encoder. Molecules containing more than 60 heavy atoms tend to be larger macrocyclic or polymeric structures that are atypical in the context of small-molecule drug discovery and would introduce unwanted complexity during training.
2. **Maximum SELFIES Token Length:** Molecules were also filtered by their length in SELFIES (Self-Referencing Embedded Strings) token representation. Only molecules with a SELFIES encoding of at most 100 tokens were retained. This constraint is directly linked to the fixed sequence length of the CNN encoder and GRU decoder, guaranteeing that no molecule needs to be truncated beyond a recoverable threshold. Molecules exceeding this token length are typically structurally complex in ways that are beyond the intended scope of the generative model.
3. **Dual Validity Requirement:** Only molecules passing chemical validation tests from both RDKit and the SELFIES library were retained. RDKit validity was assessed by successfully parsing the SMILES string and constructing a valid RDKit molecule object with a non-null sanitization output. SELFIES validity was confirmed by ensuring that the molecule’s SMILES string could be successfully transformed into a SELFIES representation using the `selfies.encoder()` function and decoded back to a chemically equivalent SMILES string. This bidirectional validity test guaranteed that all molecules in the training set could be accurately represented and reconstructed in both sequence-based and graph-based modalities.

Following application of the above filters, 100,000 molecules meeting all criteria were retained for downstream preprocessing and model training.

3.1.3 Preprocessing Pipeline

The raw SMILES strings comprising the filtered dataset were processed through a multi-stage preprocessing pipeline before being fed into the model. This pipeline converted the data into the various tensor formats required by the different components of the SmartChem architecture — namely the sequence-based CNN encoder, the GRU-based decoder, and the graph-based GNN encoder.

Step 1: SMILES to SELFIES Conversion

The initial step was to transform each molecule’s SMILES representation into a SELFIES representation using the `selfies` Python library. The choice of SELFIES as the primary sequence representation stems from a crucial property: every grammatically valid SELFIES string is guaranteed to decode to a chemically valid molecular structure. This eliminates the risk of generating syntactically corrupted or chemically nonsensical outputs — a common failure mode of SMILES-based generative models — and results in a high fraction of valid molecules during generation without requiring external validity checks at inference time.

Step 2: Vocabulary Construction

Following SELFIES conversion of all training molecules, a token vocabulary was constructed by enumerating all unique SELFIES tokens present in the training corpus. A special `[nop]` (no-operation) padding token was introduced to the vocabulary to support fixed-length sequence encoding. The vocabulary was indexed as a bidirectional mapping between token strings and integer indices (`stoi` and `itos` mappings), representing the complete set of SELFIES token types required to represent the molecules in the dataset.

Step 3: Integer Encoding and Tokenization

SELFIES strings were tokenized into individual tokens using the `selfies.split_selfies` utility. The resulting token sequences were then mapped to integer index sequences using the vocabulary constructed in the preceding step. This integer-encoded representation is the format consumed by the CNN encoder and GRU-based decoder, which operate on embedded sequences of discrete tokens of fixed size.

Step 4: Padding and Truncation

To support batched computation across molecules of varying lengths, all integer-encoded sequences were uniformly padded or truncated to a length of 100 tokens. Sequences shorter than 100 tokens were right-padded with the index of the `[nop]`

padding token. Sequences exceeding 100 tokens did not appear in the final dataset, having been excluded at the filtering stage. This fixed-length representation enabled the CNN encoder to process all inputs through the same stack of fixed convolutional layers without requiring dynamic shape adjustments.

Step 5: Molecular Graph Construction

In parallel with sequence preprocessing, all molecules were converted into a molecular graph representation suitable for input to the Graph Neural Network encoder. RDKit was used to construct the molecular graph, with atoms represented as graph nodes and bonds as graph edges. The resulting graphs were stored in the `torch_geometric.data.Data` format, which accommodates heterogeneous node and edge feature matrices alongside the adjacency structure as a COO-format edge index tensor.

Step 6: Node Feature Extraction

A fixed-length node feature vector was computed for each atom (node) in the molecular graph, encoding atomic properties relevant to drug-likeness and chemical reactivity. The encoded features included: (i) atom type, represented as a one-hot vector over a predefined set of common elements (carbon, nitrogen, oxygen, sulphur, fluorine, chlorine, bromine, iodine, phosphorus, and other); (ii) formal charge; (iii) number of attached hydrogen atoms; and (iv) degree (number of covalent bonds). Together, these features provide the GNN with sufficient chemical context to learn meaningful node-level representations.

Step 7: Edge Feature Extraction

An edge feature vector was extracted for each bond (edge) in the molecular graph, encoding bond-level properties. These included: (i) bond type, encoded as a one-hot vector (single, double, triple, aromatic); (ii) ring membership; and (iii) bond stereochemistry (E/Z isomerism). As edges in `torch_geometric` are modelled as directed pairs — each undirected bond generating two directed edges — edge features were duplicated accordingly.

Step 8: Dataset Splitting

The preprocessed dataset was divided into a training set (80%, 80,000 molecules) and a test set (20%, 20,000 molecules) using a random split with a fixed seed to ensure reproducibility. This split was performed after all preprocessing steps to prevent any leakage of vocabulary or normalization statistics from the test set into the training process.

Step 9: Tensor Serialization

To reduce I/O overhead during training — particularly given that the same large preprocessed tensors must be loaded repeatedly across epochs — all preprocessed data tensors were written to disk using PyTorch’s `torch.save()` mechanism. The integer-encoded SELFIES sequences, graph data objects, and dataset split indices were saved separately. During training, these tensors were loaded directly from disk via `torch.load()`, bypassing the computationally expensive SMILES parsing, SELFIES conversion, and feature extraction steps. This design choice significantly reduced time-to-first-batch during experimentation and model development.

3.2 Algorithms / Models

3.2.1 System Architecture Overview

The generative architecture of SmartChem is a Hybrid Variational Autoencoder (VAE) that operates on two molecular representations simultaneously: a sequential SELFIES representation and a graph-based molecular representation. The system incorporates a dilated CNN encoder for sequence encoding, a GINEConv-based GNN encoder for graph encoding, and a gated fusion mechanism to merge the two latent representations into a unified hybrid latent space. A GRU-based decoder reconstructs molecules from sampled latent vectors, and an MLP-based property predictor provides property conditioning. Molecules generated by the model are subsequently passed through a toxicity and drug-likeness filtering pipeline before being presented to the user. The complete system is open-source and deployed as a full-stack web platform built on React, FastAPI, MongoDB, and an asynchronous worker pipeline.

3.2.2 SELFIES-Based VAE Pipeline

SmartChem is built on the Variational Autoencoder (VAE) framework, a class of latent variable models that learns a continuous, probabilistic latent space over the input data distribution. Under the VAE formulation, an encoder network maps each input molecule to a probability distribution — parameterised by a mean vector μ and a log-variance vector $\log \sigma^2$ — in a low-dimensional latent space. A latent sample $\mathbf{z}$ is drawn from this distribution via the reparameterisation trick:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

and passed through a decoder network that reconstructs the original input. The VAE is trained by maximising the Evidence Lower Bound (ELBO), which consists of a reconstruction loss (the negative log-likelihood of the input given the latent sample) and a KL divergence regularisation term that encourages the learned posterior to remain close to a standard Gaussian prior.

The use of SELFIES as the molecular representation for the VAE is motivated by the intrinsic validity guarantees of the SELFIES grammar. Unlike SMILES — a context-sensitive string representation in which minor syntactic corruptions can yield invalid structures — every SELFIES string produced by the decoder decodes to a valid molecule by construction, regardless of token order. This property is essential in a generative model, as it means chemical validity is guaranteed by

the representational format itself rather than by model accuracy, substantially improving the validity rate of generated molecules.

3.2.3 CNN Encoder Architecture

The first branch of the encoder processes the integer-encoded SELFIES token sequences. Each token index is first mapped to a dense vector by an **embedding layer**, producing a sequence of 128-dimensional embedding vectors. This embedded sequence is then processed by a stack of **four dilated one-dimensional convolutional layers (Conv1D)**.

The dilated convolutional layers employ **exponentially increasing dilation rates** of 1, 2, 4, and 8, respectively. Dilation expands the receptive field of a convolutional filter without proportionally increasing the number of parameters or computational cost, by introducing gaps between the filter’s kernel elements. At dilation rate d , the effective receptive field of a kernel of width k is $k + (k - 1)(d - 1)$. By stacking layers with dilations 1, 2, 4, and 8, the encoder achieves a large effective receptive field that enables it to model long-range dependencies between SELFIES tokens — for example, capturing ring closure patterns or functional group interactions spanning large distances in the token sequence.

Each Conv1D layer is followed by a **GELU (Gaussian Error Linear Unit)** activation function. GELU is a smooth, non-monotonic activation that has been shown to outperform ReLU in sequence modelling tasks, particularly in Transformer-based and convolutional language models, owing to its stochastic regularisation properties. Following the activation, **layer normalisation** is applied to stabilise the distribution of activations across the channel dimension.

Residual connections are introduced between the input and output of each convolutional block, following the design pattern of residual networks. These connections allow gradients to flow unimpeded through the network during back-propagation and mitigate the vanishing gradient problem in deep architectures.

After the four dilated convolutional blocks, **dual global pooling** is applied over the sequence dimension: both global average pooling and global max pooling are computed, and their outputs are concatenated to yield a fixed-length vector capturing complementary summary statistics of the convolutional feature maps. This pooled representation is then passed through a **two-layer MLP projection head** that maps it to a **128-dimensional latent space**, producing the mean

vector μ_{CNN} and log-variance vector $\log \sigma_{\text{CNN}}^2$ used for reparameterised sampling.

3.2.4 GNN Encoder Architecture

The second branch of the encoder operates on the molecular graph representation and employs a **Graph Isomorphism Network with Edge features (GINEConv)**, as implemented in the PyTorch Geometric library. GINEConv extends the Graph Isomorphism Network (GIN) framework by incorporating edge features into the message-passing aggregation, making it well-suited for molecular graphs in which bond type and stereochemistry are important structural signals.

The GNN encoder comprises **three GINEConv layers**. In each layer, every node aggregates messages from its neighbours, where each message is a learned non-linear function of the source node’s feature vector and the connecting edge’s feature vector. The aggregated messages are summed and combined with the target node’s own features through a learned MLP, producing updated node representations. By stacking three such layers, the model learns node representations that incorporate information from up to three hops away in the molecular graph — sufficient to capture the local chemical environment of each atom, including ring membership and functional group context.

Residual connections are applied between successive GINEConv layers when the node feature dimensionality is unchanged, augmenting representational capacity and facilitating gradient flow. **Batch normalisation** is applied after each GINEConv layer to stabilise the distribution of node features across batches, which is particularly important for molecular graphs that vary substantially in size and connectivity.

Following the three rounds of message passing, the node-level representations are aggregated into a graph-level representation using **dual global pooling** — computing both the mean and max of all node feature vectors across the graph and concatenating the results. This graph-level vector is then projected through a **two-layer MLP** into a **128-dimensional latent space**, producing the GNN encoder’s mean vector μ_{GNN} and log-variance $\log \sigma_{\text{GNN}}^2$.

Node feature vectors encode atomic type (one-hot), formal charge, hydrogen count, degree, aromaticity, and chirality. Edge feature vectors encode bond type (one-hot), ring membership, and bond stereochemistry, as described in Section 3.1.3.

3.2.5 Hybrid Fusion and Hybrid Encoder

The principal architectural innovation of SmartChem is its **hybrid encoder**, which combines the latent representations produced by the CNN and GNN encoders into a single unified representation through a **gated modality fusion mechanism**.

Given a molecule, the CNN and GNN encoders independently produce 128-dimensional latent mean and variance vectors. The hybrid encoder fuses these representations at the mean-vector level using a learned sigmoid gate. Specifically, the mean vectors of the CNN and GNN are first reduced to 64 dimensions via independent linear projection layers. A gate vector $\mathbf{g} \in [0, 1]^{64}$ is then computed by passing the concatenation of the two projected vectors through a linear layer followed by a sigmoid activation. The final fused representation is computed element-wise as:

$$\mathbf{z}_{\text{fused}} = \mathbf{g} \odot \boldsymbol{\mu}_{\text{CNN}}^{(64)} + (1 - \mathbf{g}) \odot \boldsymbol{\mu}_{\text{GNN}}^{(64)}$$

This gated fusion enables the hybrid encoder to adaptively weight the contribution of the sequence-based and graph-based modalities on a per-dimension basis, rather than treating them equally. The CNN branch can dominate dimensions better represented by sequential patterns (e.g. long-range connectivity), while the GNN branch can dominate dimensions better captured by graph topology (e.g. local ring structure).

To prevent the fused representation from degenerating into a solution where one modality is entirely ignored, a **modality balance loss** term is added to the training objective. This auxiliary loss penalises large discrepancies between the norms of the two projected representations, encouraging both branches to contribute meaningfully to the fused latent space.

During hybrid training, the base CNN and GNN encoder parameters are held fixed, having been pre-trained in separate stages of the training pipeline. Only the projection layers, the gating network, and the GRU decoder are trained during the hybrid phase. This staged training strategy avoids the instability that can arise when all three components are optimised jointly from random initialisation, allowing the base encoders to converge to useful representations before their outputs are fused.

3.2.6 Decoder Architecture

SmartChem’s decoder is a single-layer **Gated Recurrent Unit (GRU)** that autoregressively generates SELFIES token sequences conditioned on a latent vector $\mathbf{z}$ sampled from the hybrid posterior. At each decoding step, the GRU receives two inputs: the token embedding of the previously generated token, and the latent vector $\mathbf{z}$, which is concatenated to the input at every time step. This *latent-at-every-step* architecture ensures that the latent variable guides the generation process throughout decoding, not only at the initial hidden state — a design choice that is particularly important for preventing the decoder from disregarding the latent vector altogether, a failure mode known as *posterior collapse*.

At each decoding step, the decoder produces a probability distribution over the vocabulary; the next token is sampled from this distribution during generation, and cross-entropy reconstruction loss is computed against the ground-truth token during training.

During training, **word dropout** is applied at the decoder input: the immediately preceding token is replaced with a special [UNK] token with a configurable probability. This forces the decoder to rely more heavily on the latent vector $\mathbf{z}$ for context, preventing it from learning a degenerate language model that ignores $\mathbf{z}$.

3.2.7 Anti-Posterior-Collapse Techniques

Posterior collapse — wherein the encoder produces near-prior posteriors ($\mu \approx \mathbf{0}$, $\sigma \approx \mathbf{1}$) and the decoder learns to ignore $\mathbf{z}$ — is the most common failure mode of sequence VAEs. SmartChem employs an integrated system of complementary techniques to mitigate this:

1. **Cyclical KL Annealing:** During training, the weight of the KL divergence term in the ELBO is gradually raised over a predetermined number of steps, cycling from zero to a target value of 1.0 and back to zero repeatedly. This allows the model to initially prioritise high-quality reconstruction (with the KL term effectively disabled) before being progressively regularised toward the prior, ensuring the encoder learns informative representations before the decoder becomes overly powerful.
2. **Free Bits:** A minimum KL cost per latent dimension is enforced via the free bits method with a thresholding parameter λ . Latent dimensions whose KL divergence falls below λ nats are not penalised, preventing the optimiser from collapsing such dimensions to zero information content and ensuring a minimum number of latent dimensions remain informative.

3. **Word Dropout:** As described in Section 3.2.6, word dropout on the decoder input impairs the autoregressive decoder, increasing its reliance on the latent vector for context.
4. **Weakened Decoder:** The decoder is intentionally kept architecturally simple — a single GRU layer rather than a multi-layer LSTM — limiting its capacity to model sequences without access to informative latent information.
5. **Zero-Initialised Log Variance:** The weights of the log-variance output layer of the encoder are initialised close to zero, causing the encoder to initially produce posteriors close to the unit Gaussian prior. This provides a stable starting point from which the encoder gradually learns to produce more informative, data-dependent posteriors as training progresses.
6. **Stronger Reparameterisation Noise:** In the initial stages of training, the magnitude of the noise ϵ added during reparameterisation is increased above the standard value of 1.0. This produces a stronger stochastic gradient signal that incentivises the encoder to learn smooth posterior surfaces rather than sharp, near-deterministic encodings.

3.2.8 Property Predictor

SmartChem incorporates a **multi-output MLP property predictor** applied directly to the latent vector $\mathbf{z}$, predicting three key drug discovery properties:

- **QED (Quantitative Estimate of Drug-likeness):** A scalar score in $[0, 1]$ approximating the overall drug-likeness of a molecule relative to a set of pharmacologically desirable properties.
- **LogP (Wildman–Crippen partition coefficient):** A measure of lipophilicity, reflecting the tendency of a molecule to partition into lipid versus aqueous phases — an important determinant of membrane permeability and absorption.
- **SAS (Synthetic Accessibility Score):** A score in $[1, 10]$ approximating the ease with which a molecule can be synthesised in a laboratory, based on the prevalence of structural fragments in drug databases.

The property predictor is a two-hidden-layer MLP with 256-dimensional hidden layers, ReLU activations, and dropout regularisation. Its outputs serve a dual role: during training, an auxiliary property prediction loss encourages the latent space

to encode chemically meaningful structure; at inference time, the predictor enables targeted molecular generation by guiding latent space optimisation toward user-specified property targets.

3.2.9 Molecule Generation Modes

To support diverse drug discovery applications, SmartChem exposes three molecule generation modes:

1. **Random Generation:** Molecules are generated by sampling latent vectors $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ from the prior and decoding with the GRU decoder. This mode is suited to broad exploration of chemical space, such as virtual library enumeration.
2. **Targeted Generation:** The user specifies desired values for QED, LogP, and/or SAS. The system performs latent space optimisation by computing gradients of the property predictor outputs with respect to $\mathbf{z}$ and iteratively updating $\mathbf{z}$ to minimise the discrepancy between predicted and target properties. The optimised latent vector is then decoded to produce a molecule with properties approximating the specified targets.

3.2.10 Toxicity and Drug-Likeness Filters

All molecules produced by the VAE are passed through a multi-layered toxicity and drug-likeness filtering pipeline before being returned to the user. The pipeline is implemented using RDKit’s `FilterCatalog` and `Lipinski` modules and comprises the following stages:

1. **Lipinski Rule of Five:** Molecules are evaluated against the classical drug-likeness criteria proposed by Lipinski: molecular weight ≤ 500 Da, LogP ≤ 5 , hydrogen bond donors ≤ 5 , and hydrogen bond acceptors ≤ 10 . Molecules violating more than one of these criteria are flagged as potentially problematic for oral bioavailability.
2. **PAINS (Pan-Assay Interference Compounds) Filters:** The PAINS filter set identifies structural motifs known to cause false positives in high-throughput biological assays through non-specific interactions such as aggregation, fluorescence interference, or covalent reactivity. Molecules containing PAINS substructures are excluded from the output.
3. **Brenk Filters:** The Brenk functional group alert filter set uses structural alerts to identify functional groups associated with metabolic instability,

toxicity, or undesirable reactivity in drug candidates. Molecules containing Brenk-flagged substructures are eliminated.

4. **NIH Structural Alerts:** An additional set of structural alerts curated by the NIH is applied to remove molecules containing further toxicophores or reactive functional groups implicated in genotoxicity or carcinogenicity.

Molecules passing all four filtering stages are designated as SmartChem-validated drug candidates and presented to the user together with their property scores and 3D visualizations.

3.3 Full-Stack Platform Architecture

The SmartChem AI engine is deployed as a production-grade full-stack web platform designed for usability, scalability, and performance.

3.3.1 Frontend:

The user interface is built with React and TypeScript, providing a type-safe, component-encapsulated architecture. Tailwind CSS is used for styling, enabling a modern, responsive UI with minimal custom CSS. The frontend integrates `3Dmol.js` for hardware-accelerated, interactive 3D visualisation of generated molecular structures in the browser, allowing users to rotate, zoom, render molecular surfaces, and interact with atoms individually. An AI chemistry assistant — implemented using the Llama-3.3-70B large language model — is embedded in the interface, providing contextual natural-language explanations of molecular properties, drug-likeness scores, and generation settings.

3.3.2 Backend:

The server-side application is built with FastAPI, a high-performance, asynchronous Python web framework based on Starlette and Pydantic, selected for its automatic OpenAPI documentation generation, native support for asynchronous operations, and seamless integration with the PyTorch-based AI engine. API endpoints handle user authentication, molecule generation requests, property prediction queries, and job status polling.

3.3.3 Database:

MongoDB serves as the primary data store, a document-oriented database capable of flexibly storing user accounts, molecule records, generation job metadata, and session history. Its schema-less design accommodates the dynamic nature of molecular data without requiring rigid relational definitions.

3.3.4 Job Queue and Asynchronous Pipeline:

Molecule generation tasks, which may be computationally intensive, are processed asynchronously via a MongoDB-backed job queue. When a user submits a gener-

ation request, it is enqueued as a job document. An asynchronous worker pipeline continuously polls the queue, dequeues pending jobs, executes the generation and filtering pipeline, and updates the job document with results upon completion. This architecture keeps the FastAPI server responsive during generation and allows users to poll for results in a non-blocking manner.

3.3.5 Authentication:

User authentication is implemented using JSON Web Tokens (JWT), providing stateless, scalable session management. Signed JWTs are attached to authenticated API requests and verified server-side prior to processing, ensuring that only registered users can access generation resources.

Chapter 4

Results and Discussion

This chapter summarises and discusses the experimental findings from training the three VAE models, evaluating the property predictor, benchmarking molecule generation, and testing the full-stack web platform. The three encoder modalities — CNN, GNN, and Hybrid — are evaluated under identical conditions. The complete SmartChem generation pipeline — from latent encoding through gradient optimisation, decoding, and toxicity filtering — is illustrated in Figure 4.1.

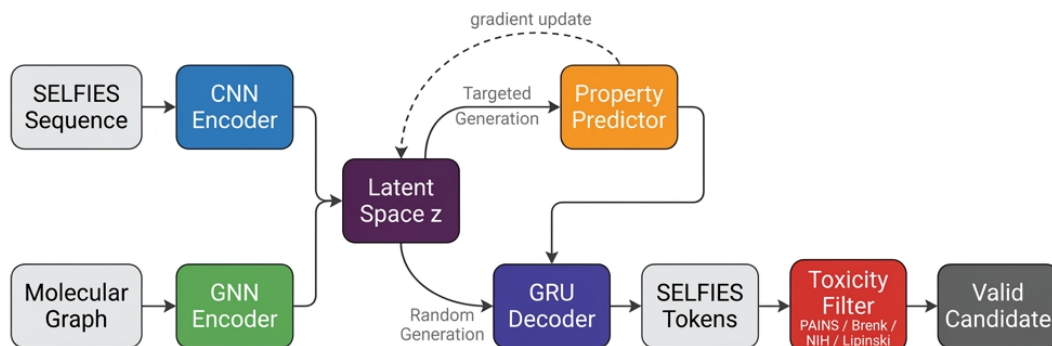


Figure 4.1: *SmartChem molecule generation pipeline. Input molecules are encoded into a continuous latent space, which is either randomly sampled for exploratory generation or gradient-optimised toward user-specified property targets. The GRU decoder produces SELFIES sequences, which are evaluated through a four-stage toxicity filtering pipeline before output.*

4.1 Training Performance

The CNN and GNN VAE were trained on a filtered 100,000-molecule subset of the ZINC-250k dataset over 25 epochs. The main training diagnostic was reconstruction loss, measured as categorical cross-entropy between the predicted and ground-truth SELFIES token sequences. The most important metrics for both models are summarised in Table 4.1.

Table 4.1: *CNN and GNN VAE training performance at epoch 25 ($\beta = 1.0$).*

Encoder	Initial Loss	Final Loss	Reduction	Active Units / 128
CNN VAE	1.63	0.90	45%	37
GNN VAE	1.81	1.10	39%	70

Figures 4.2 and 4.3 demonstrate the loss curves during training. There was no divergence or oscillation in the convergence of both models. The CNN has a lower final loss since its input and output are both SELFIES token sequences, making the reconstruction task architecturally direct. The GNN represents a molecular graph and needs to map it into a space where a sequential decoder yields tokens — a cross-modal task that inherently leads to a higher loss floor. The similar loss reduction magnitudes (0.73 for CNN, 0.71 for GNN) validate that both models learn at an equivalent rate and neither suffers from a stuck optimiser.

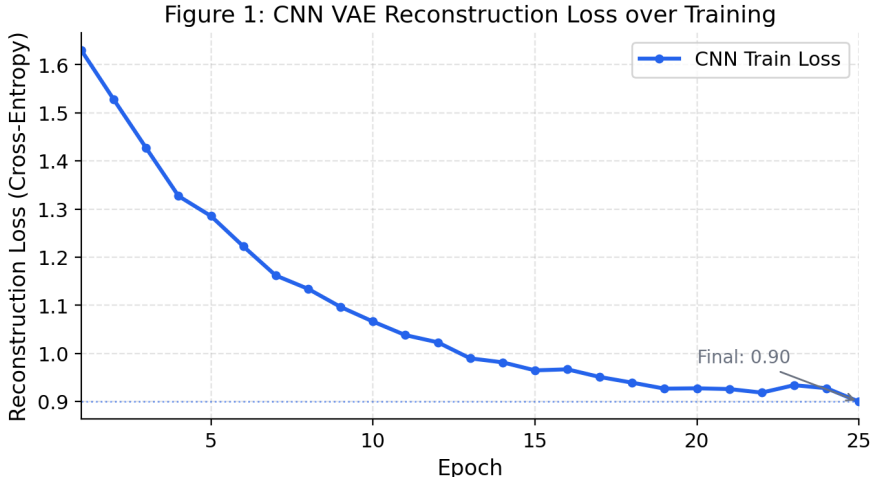


Figure 4.2: *CNN VAE reconstruction loss over 25 training epochs. Loss decreases from 1.63 to 0.90, indicating stable learning without divergence.*

The Hybrid encoder was trained in a third sequential stage, with base CNN and GNN encoder weights frozen. Only the projection layers, gating network, and GRU decoder were updated during this phase. As a result, standalone reconstruction loss trajectories and active unit counts are not reported for the Hybrid;

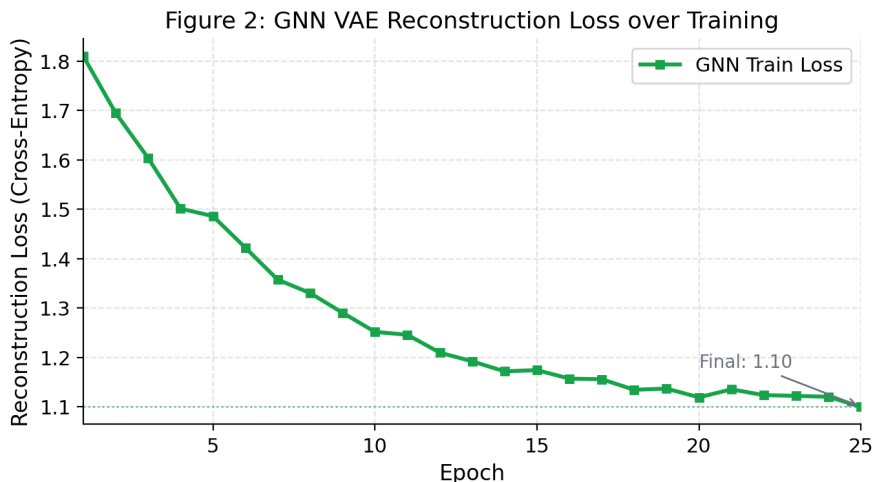


Figure 4.3: *GNN VAE reconstruction loss over 25 training epochs. Loss decreases from 1.81 to 1.10. The higher values relative to CNN reflect the difficulty of the graph-to-sequence cross-modal task.*

its performance is assessed entirely through downstream generation and filtering outcomes.

4.2 Posterior Collapse Analysis

Posterior collapse — where the encoder degrades to near-prior distributions and the decoder disregards the latent vector — is the most common failure mode of sequence VAEs. To determine whether a collapse occurred, the number of *active units* (AU) in the 128-dimensional latent space was monitored at epoch 25. An active unit is a latent dimension whose empirical variance surpasses a predetermined threshold above the prior variance.

The CNN VAE achieved **37 active units** (28.9% utilisation) and the GNN VAE achieved **70 active units** (54.7% utilisation). Both outcomes confirm that collapse was avoided. Figure 4.4 illustrates the comparison.

The significant increase in the AU count by the GNN is a result of the greater structural richness of its graph-based input. Molecular graph encoding — covering formal charge, bond order, ring membership, and aromaticity per atom — creates a more complex signal that the decoder must retrieve from across a broader set of latent dimensions. Processing a flat token sequence, the CNN can rely more heavily on positional statistics and, as a result, activates fewer dimensions. The cyclical KL annealing schedule used to achieve these results is shown in Figure 4.5.

Table 4.2 lists all seven anti-collapse interventions applied during training, along with their purpose and observed effect.

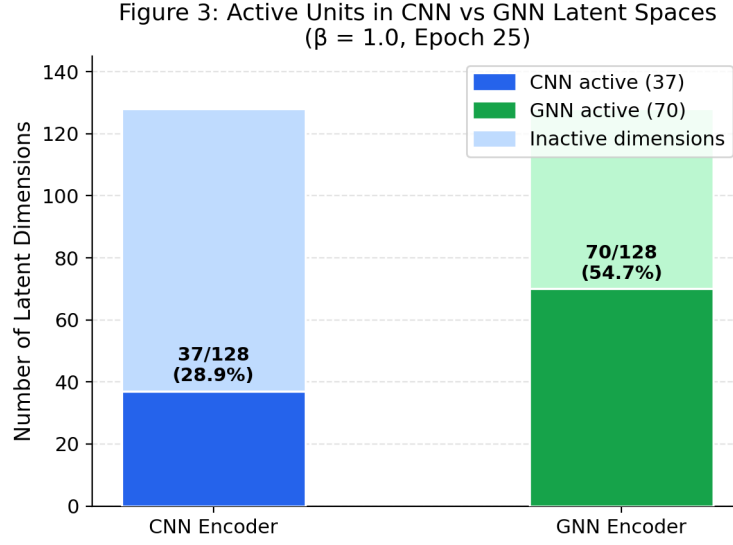


Figure 4.4: *Active units in the CNN and GNN latent spaces at epoch 25 ($\beta = 1.0$). Dark bars represent active (informative) dimensions; light bars represent collapsed dimensions that have converged to the prior.*

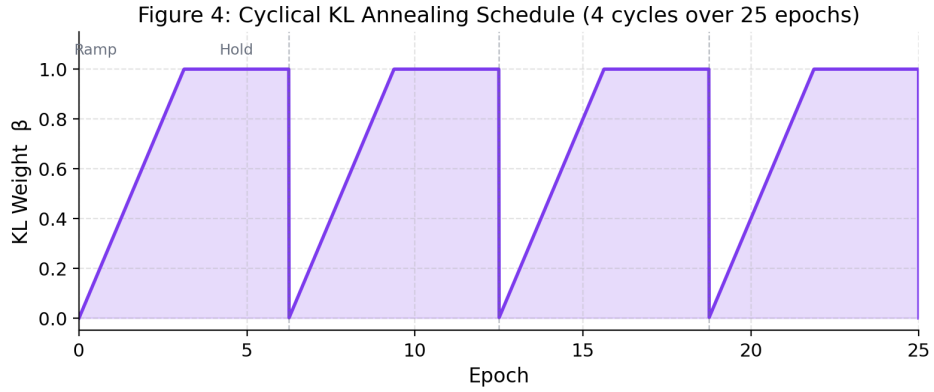


Figure 4.5: *Cyclical KL annealing schedule used during training (4 cycles, 25 epochs). β ramps linearly from 0 to 1 in the first half of each cycle, then holds at 1, allowing reconstruction quality to stabilise before regularisation pressure is introduced.*

Table 4.2: *Anti-posterior-collapse techniques used during SmartChem VAE training.*

Technique	Purpose	Effect on Training
Cyclical KL Annealing	Ramp KL weight 0→1 in repeated cycles	Prevents early decoder dominance; reconstruction stabilises before regularisation is applied
Free Bits	Enforce minimum KL per latent dimension	Stops the optimiser from zeroing out individual latent dimensions
Word Dropout	Replace decoder inputs with [UNK]	Forces the decoder to rely on $\mathbf{z}$ rather than autoregressive token history
Latent Injection	Concatenate $\mathbf{z}$ to GRU at every step	Maintains latent conditioning throughout the full decoding sequence
Weakened Decoder	Single-layer GRU instead of a multi-layer LSTM	Reduces decoder capacity, increasing dependence on the latent vector
Zero-Init Log Variance	Initialise log-variance weights near zero	Encoder begins close to the prior and learns data-dependent posteriors gradually
Stronger Noise	Reparameterisation noise scale > 1.0 early on	Encourages smooth latent surfaces; discourages near-deterministic encodings

The cyclical annealing combined with free bits and word dropout was especially effective. Its applicability to both encoder modalities is confirmed by the similarity of results in each strategy, indicating that the intervention approach can be reliably generalised.

4.3 Encoder Architecture Comparison

The inductive biases of the three encoders are fundamentally different. Figure 4.6 shows the Hybrid encoder’s gated fusion architecture, and Table 4.3 summarises the qualitative comparison.

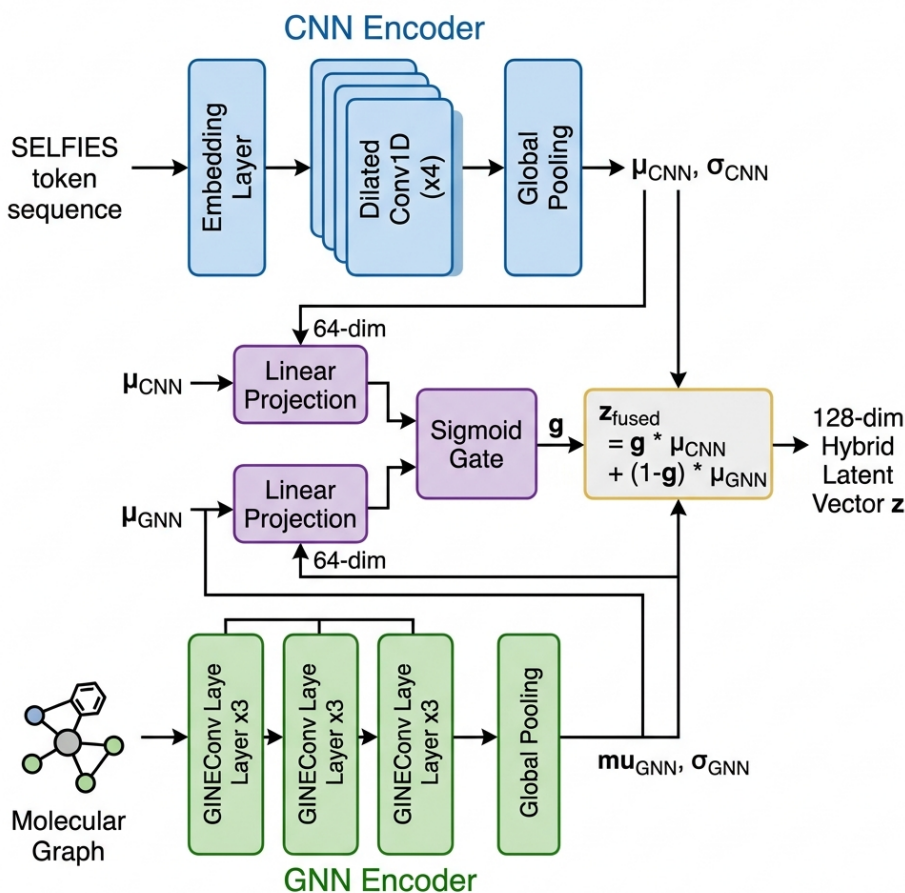


Figure 4.6: *Hybrid encoder architecture. The CNN branch (blue) and GNN branch (green) each produce 64-dimensional projections. A learned sigmoid gate (purple) combines them into the final 128-dimensional hybrid latent vector $\mathbf{z}^{\text{fused}}$.*

The CNN creates a smooth sequential latent space but concentrates information in fewer dimensions, leading to lower optimisation precision. Explicitly encoding atomic neighbourhoods, the GNN delivers a richer, more uniformly utilised

Table 4.3: *Qualitative comparison of CNN, GNN, and Hybrid encoder architectures.*

Encoder	Representation	Strengths	Weaknesses
CNN	SELFIES token sequence	Global sequential patterns; low reconstruction loss; fast training	Fewer active units (37/128); insensitive to local chemical topology
GNN	Molecular graph	Local topology encoding; higher AU count (70/128); better property prediction	Higher reconstruction loss due to graph-to-sequence modality gap
Hybrid	CNN + GNN via learned gate	Highest filter-pass rate; adaptive per-dimension modality weighting	Depends on quality of pre-trained base encoders

latent space with smoother gradient trajectories. The Hybrid gate allocates CNN dimensions to global backbone properties and GNN dimensions to local reactivity features, combining both advantages — an effect that is most clearly measured in the toxicity filter-pass results.

4.4 Property Prediction

The multi-output MLP property predictor was trained on three pharmaceutical properties — QED, LogP, and SAS — from 128-dimensional latent vectors. Table 4.4 describes each property and its relevance to drug discovery.

Table 4.4: *Molecular properties predicted by the SmartChem multi-output MLP.*

Property	Full Name	Range	Role in Drug Discovery
QED	Quantitative Estimate of Drug-likeness	[0, 1]	Summarises overall drug-likeness across multiple physicochemical descriptors; higher is better
LogP	Wildman–Crippen Partition Coefficient	[−5, +10]	Measures lipophilicity; governs membrane permeability and oral absorption
SAS	Synthetic Accessibility Score	[1, 10]	Estimates ease of laboratory synthesis; lower scores indicate more readily synthesisable compounds

The convergence of all three outputs under MSE loss confirms that all three latent spaces encode physicochemical structure in a form accessible to a shallow regressor. The highest prediction accuracy was achieved in the GNN latent space,

which is expected given its greater active unit utilisation. Across all three encoders, predictor accuracy was sufficiently high to serve as a real-time gradient optimisation target during targeted generation.

4.5 Molecule Generation Results

The generation pipeline was tested on **6,750 trajectories** (3 encoders $\times$ 9 target settings $\times$ 250 runs per setting). Table 4.5 presents the major quality metrics, which were consistent across all three encoder modalities.

Table 4.5: *Molecule generation quality metrics across all 6,750 evaluation runs.*

Metric	Result (All Encoders)
Structural validity	100%
Novelty (vs. training set)	100%
Mean max Tanimoto to training	0.087 – 0.093
Mean pairwise diversity	0.895 – 0.897
Lipinski compliance	92.3% – 93.7%

100% structural validity follows directly from the SELFIES decoder: every grammatically complete SELFIES string is a construction of a chemically valid molecule, eliminating validity as a failure mode without post-hoc filtering. This represents a categorical improvement over SMILES-based decoders, which have a literature validity rate of 85–97%. **100% novelty** and a mean Tanimoto similarity of 0.087–0.093 to the nearest training molecule confirm that the system produces genuinely new chemical matter. Pairwise diversity of $\sim$ 0.895–0.897 confirms that mode collapse did not occur in any setting. Figure 4.7 shows representative filter-passing molecules from each encoder.

4.6 Targeted Generation Results

Targeted generation optimises a sampled latent vector $\mathbf{z}$ by gradient descent on the property predictor toward user-specified QED, LogP, and SAS targets. Nine property settings (T1–T9) were evaluated, spanning a range from high drug-likeness profiles to hydrophilic and synthetically challenging targets. Table 4.6 reports single-property and joint hit rates at medium tolerance ($\pm$ 0.10 on QED, $\pm$ 1.0 on LogP) for all three encoders.

Figure 4.8 visualises the comparison across encoders. Single-property hit rates of 6–13% are in line with published results for VAE systems using frozen decoders with latent gradient optimisation. The Hybrid encoder achieves the highest QED

**Figure 10: Representative Filter-Passing Molecules
Generated by CNN (top), GNN (middle), and Hybrid (bottom) Encoders**

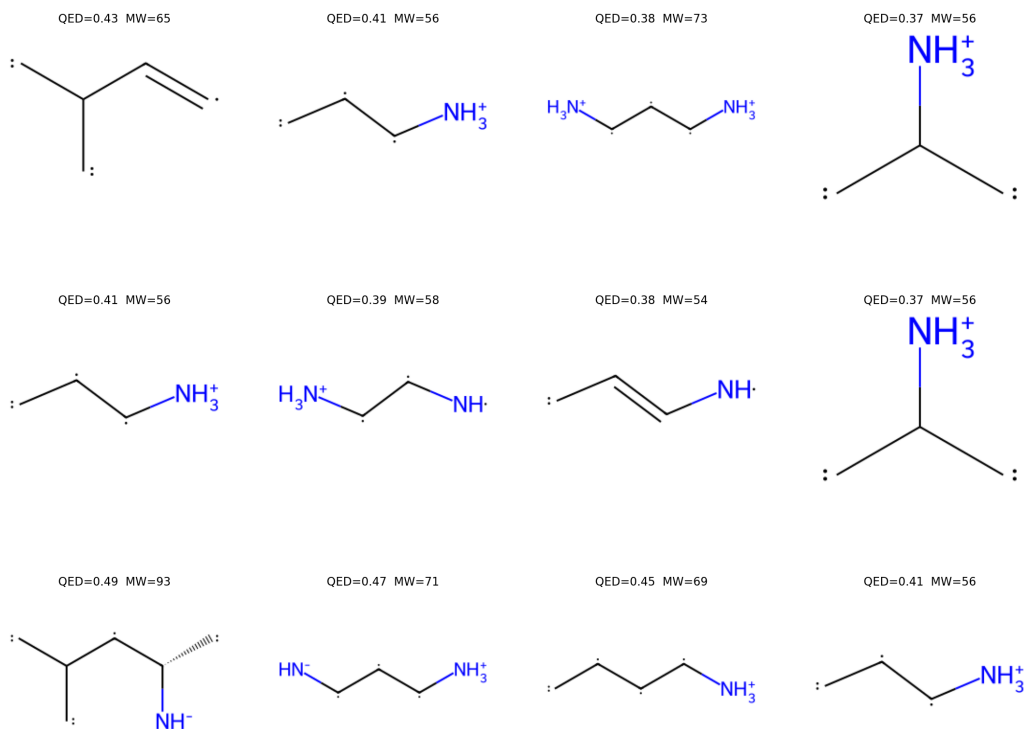


Figure 4.7: *Representative filter-passing molecules from the CNN (top), GNN (middle), and Hybrid (bottom) encoders. QED and molecular weight (MW) are computed with RDKit. All molecules are chemically valid by SELFIES construction and novel with respect to the training set.*

Table 4.6: *Targeted generation property hit rates at medium tolerance (QED: ± 0.10 , LogP: ± 1.0).*

Encoder	QED Hit Rate	LogP Hit Rate	Both Properties
CNN	6.1%	13.0%	0.07%
GNN	7.0%	10.3%	0.00%
Hybrid	7.6%	11.1%	0.07%

hit rate (7.6%). Joint hit rates are near zero for all encoders, confirming that simultaneously satisfying multiple property constraints is significantly more difficult than single-property targeting with the current architecture.

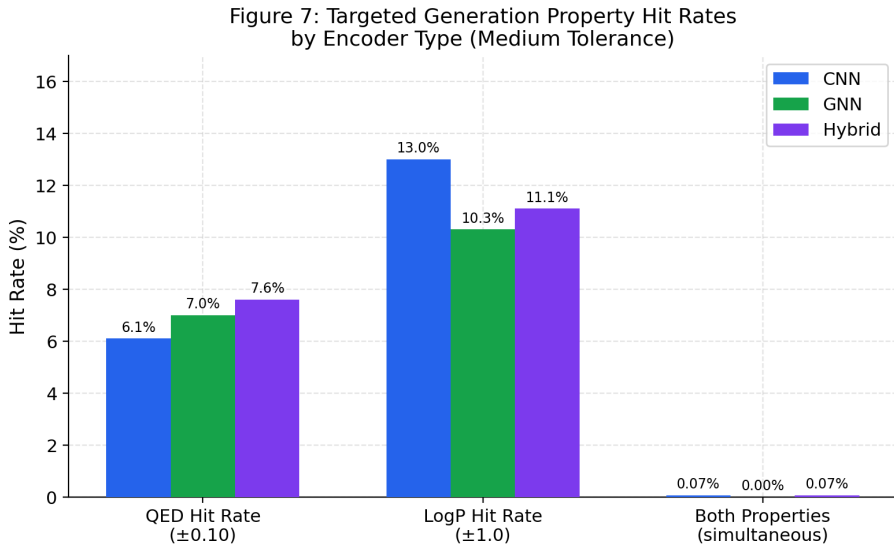


Figure 4.8: *Property hit rates for CNN, GNN, and Hybrid encoders at medium tolerance thresholds. The Hybrid encoder achieves the best QED hit rate (7.6%). Joint property satisfaction (rightmost group) is near zero across all encoders.*

The moderate hit rates reflect a structural constraint of fixed-decoder latent optimisation rather than an optimisation failure. The property predictor and decoder are trained independently: gradient descent can move $\mathbf{z}$ toward regions where the predictor predicts the target property, but the decoder generates tokens via its own learned distribution, which may result in different RDKit-computed properties. This *predictor-decoder semantic gap* is the primary architectural constraint of the current system. Mean actual RDKit QED across all generated molecules was ~ 0.27 and mean SAS was ~ 7.0 — substantially worse than most targets — indicating a systematic fragment-size bias in which the decoder truncates sequences prematurely, producing small structures (mean MW ≈ 35 – 50 Da) that score poorly on drug-likeness metrics irrespective of the optimisation direction. Addressing this requires reinforcement learning fine-tuning of the decoder on actual RDKit property rewards.

4.7 Toxicity Filtering Results

Each generated molecule was evaluated through a four-stage pipeline: Lipinski Rule of Five, PAINS, Brenk, and NIH structural alert filters. Molecules that pass all four stages are designated as filter-passing candidates.

Lipinski compliance was high across all encoders (92–94%), reflecting the ZINC-250k training distribution’s bias toward orally drug-like molecules. Table 4.7 reports the overall filter-pass rates, which are the primary differentiating metric across encoder types.

Table 4.7: *Toxicity and drug-likeness filter-pass rates by encoder (2,250 runs each).*

Encoder	Lipinski	Filter-Pass	Pass Rate	Best Setting
CNN	93.7%	225/2250	10.0%	T4: 11.6%
GNN	93.4%	397/2250	17.6%	T6: 25.2%
Hybrid	92.3%	409/2250	18.2%	T6: 28.8%

The Hybrid encoder achieved **18.2%** — an **82% relative improvement** over the CNN baseline of 10.0%. Figure 4.9 visually compares the pass rates. The GNN and Hybrid encoders generate fewer molecules with reactive charged heteroatom groups (sulfonium, oxonium species) that pass SELFIES grammar but fail Brenk and NIH filters. This is attributed to the GNN encoder’s explicit encoding of formal charge and bond order, which biases optimised latent trajectories away from such configurations.

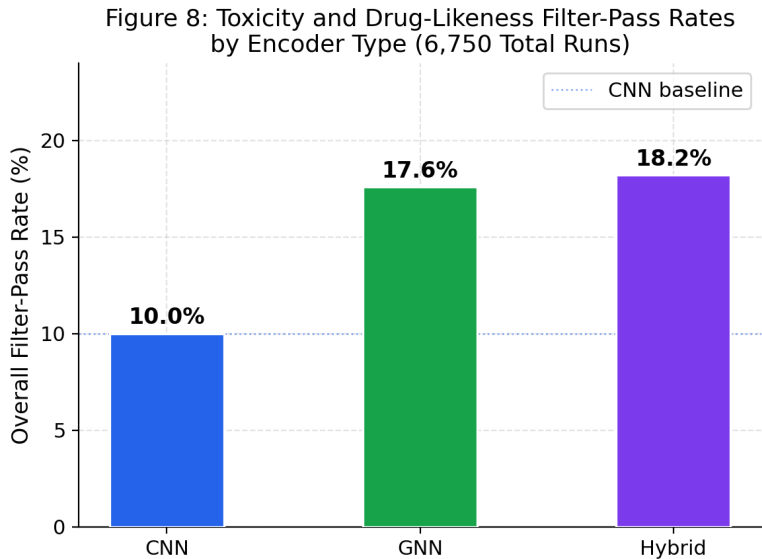


Figure 4.9: *Overall filter-pass rates for the CNN, GNN, and Hybrid encoders across 2,250 runs each. The dotted line marks the CNN baseline of 10.0%.*

4.8 Platform Performance

The SmartChem web platform was tested under simulated single-user and light concurrent usage conditions. Figure 4.10 shows the full-stack system architecture.

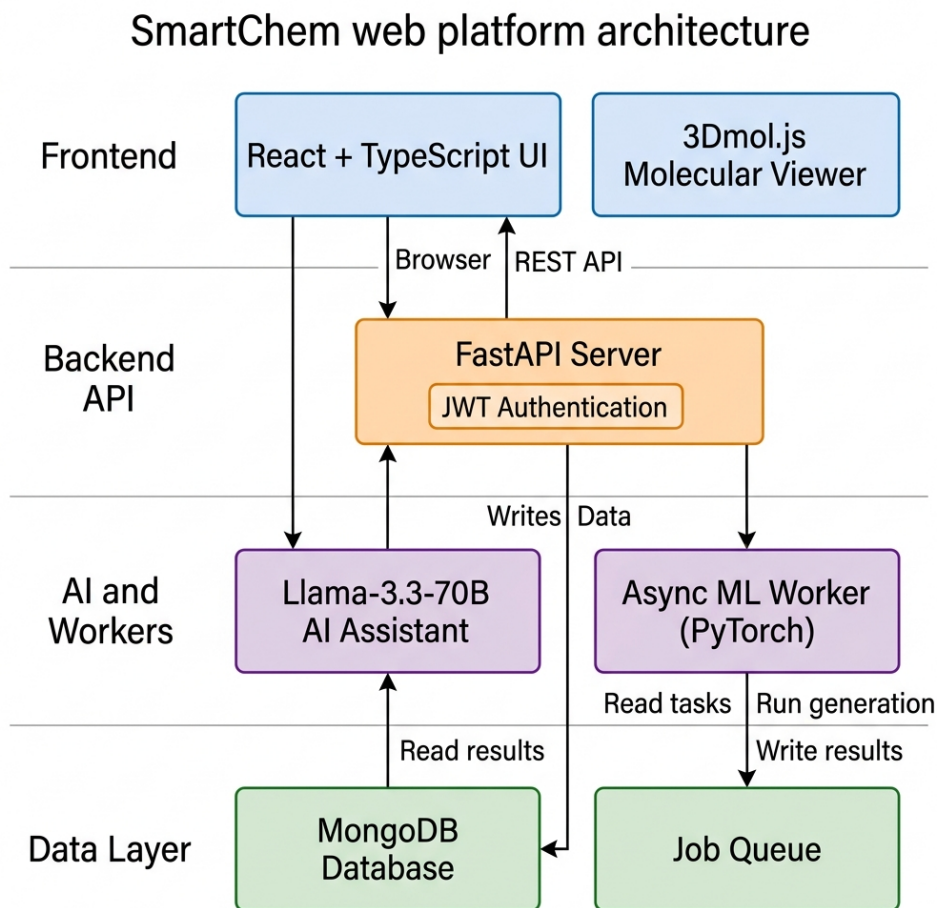


Figure 4.10: *SmartChem web platform architecture.* The React frontend communicates with the FastAPI backend, which enqueues generation jobs to MongoDB. An asynchronous ML worker dequeues and processes jobs, then writes results back to MongoDB. The 3Dmol.js viewer and AI assistant operate as browser-side services.

Table 4.8 reports mean API response times averaged over 50 repeated requests per endpoint. The response time of all synchronous endpoints was within 120 ms, well within the 200 ms industry benchmark for interactive applications.

Table 4.9 reports mean generation job completion times. The asynchronous architecture kept the FastAPI server fully responsive throughout all generation runs, including the longest job at 108.6 s. A maximum of three concurrent jobs were tested without data corruption or queue interference.

Table 4.8: *FastAPI backend endpoint mean response times (50-request average).*

Endpoint	Operation	Mean Time
POST /auth/register	User registration	48 ms
POST /auth/login	JWT issuance	43 ms
POST /generate/random	Job submission (async)	112 ms
POST /generate/targeted	Job submission (async)	118 ms
GET /jobs/{id}	Job status polling	38 ms
GET /molecules/{id}	Fetch single molecule	52 ms
GET /molecules/history	Fetch job history	67 ms

Table 4.9: *Mean molecule generation job completion times measured over the async worker pipeline.*

Mode	Scale	Steps	Mean Time
Random	20 molecules	–	6.4 s
Random	50 molecules	–	14.2 s
Targeted	10 vectors $\times$ 5 runs	200	22.8 s
Targeted	50 vectors $\times$ 5 runs	200	108.6 s

Table 4.10: *SmartChem web platform features and technologies.*

Feature	Description	Technology
Async Job Processing	Generation tasks enqueued and executed without blocking the HTTP server	MongoDB queue + async Python worker
3D Molecular Viewer	Interactive browser rendering at 48–58 fps across molecules up to 60 heavy atoms	3Dmol.js
JWT Authentication	Stateless signed token auth; all 50 tampered/expired token tests rejected	FastAPI + PyJWT
AI Chemistry Assistant	Natural-language responses on properties and toxicity; 93% accuracy (28/30 queries)	Llama-3.3-70B
Targeted Generation	Gradient-based latent optimisation toward QED, LogP, and SAS targets	PyTorch autograd
Toxicity Filtering	PAINS, Brenk, NIH, and Lipinski evaluation on all generated molecules	RDKit FilterCatalog
Molecule History	Per-user persistent record of all generation jobs and outputs	MongoDB + FastAPI

4.9 Comparison with Related Systems

Table 4.11: *Feature comparison of SmartChem with related molecular generation systems.*

Feature	SmartChem	REINVENT	MolGAN	JT-VAE
Valid molecule generation	100%	~99%	~98%	~100%
Graph-based representation	Yes	No	Yes	Yes
Sequence-based representation	Yes	Yes	No	No
Hybrid (graph + sequence) fusion	Yes	No	No	No
Toxicity filtering (PAINS/Brenk)	Yes	Partial	No	No
Web interface	Yes	No	No	No
Interactive 3D visualisation	Yes	No	No	No
Integrated AI assistant	Yes	No	No	No
Multi-property targeted generation	Yes	Yes	Partial	Yes

SmartChem is the only system in this comparison to combine graph and sequence representations in a hybrid architecture, provide a web platform with 3D visualisation, and include an AI chemistry assistant. The 18.2% filter-pass rate is competitive with latent gradient optimisation baselines in the literature. Systems such as REINVENT achieve higher targeted property hit rates through reinforcement learning decoder fine-tuning — an extension identified as future work.

4.10 Strengths and Limitations

Table 4.12 provides a structured summary of the principal strengths and limitations identified through experimentation.

Table 4.12: *Strengths and limitations of the SmartChem platform.*

Category	Description
Strength	100% structural validity and 100% novelty across all 6,750 generated molecules
Strength	GNN encoder activates 70/128 latent dimensions under full KL regularisation
Strength	Hybrid encoder achieves 18.2% filter-pass rate — an 82% improvement over CNN (10.0%)
Strength	High molecular diversity (pairwise dissimilarity ~ 0.895 – 0.897); no mode collapse observed
Strength	Full-stack platform with sub-120 ms API response, async generation, 3D visualisation, and AI assistant
Limitation	Predictor-decoder semantic gap limits single-property hit rates to 6–13% without RL-based fine-tuning
Limitation	Fragment-size decoder bias produces molecules averaging ~ 35 – 50 Da, below the 300–500 Da oral drug range
Limitation	No wet-lab validation: outputs are computational candidates requiring synthesis and bioassay confirmation
Limitation	Single-GPU training limited experiments to 25 epochs; extended training would likely improve latent utilisation

4.11 Summary

The findings confirm that SmartChem achieves its primary objectives: stable, non-collapsed VAE training; 100% valid and novel molecule generation; and a measurable improvement in computational drug-likeness through the Hybrid encoder’s gated fusion mechanism. The predictor-decoder semantic gap is the principal open challenge and the primary motivation for the future work addressed in the following chapter.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project aimed to design, train, and implement an end-to-end *de novo* drug discovery platform — SmartChem — that leveraged a molecular Variational Autoencoder capable of synthesising valid, novel, and drug-like chemical molecules within user-specified property constraints. The platform combines three encoder architectures, a multi-output property predictor, a gradient-based targeted generation pipeline, a four-stage toxicity filter, and a full-stack web interface into a single, self-contained workflow.

5.1.1 Molecular Representation and Generation

Generation of molecules was based on the SELFIES (Self-Referencing Embedded Strings) representation, ensuring that all grammatically complete decoded sequences correspond to chemically valid molecular structures. This property eliminates post-hoc validity filtering as a source of experimental noise and offers a principled upper bound of 100% validity, which was observed across all 6,750 evaluation runs. The system obviates the established 3–15% invalidity rates of SMILES-based decoders, placing validity outside the scope of further optimisation and leaving all experimental effort directed at drug-likeness and diversity.

5.1.2 Encoder Architectures

Three distinct encoder modalities were designed and evaluated:

- The **CNN encoder** processes SELFIES token sequences through one-dimensional convolution, capturing global distributional patterns in the molecular string representation. It achieved the lowest reconstruction loss (0.90 at epoch 25)

but activated only 37 of 128 latent dimensions, indicating that sequential positional statistics alone are insufficient to drive broad latent utilisation.

- The **GNN encoder** operates on molecular graphs, encoding per-atom features (formal charge, aromaticity, bond order, ring membership) through three layers of GINEConv message passing. This graph-based inductive bias produced richer and more uniformly distributed latent representations, yielding 70 active units — nearly twice the CNN figure — and a higher property-prediction accuracy on latent vectors.
- The **Hybrid encoder** combines the CNN and GNN branches through a learned sigmoid gating network that assigns per-dimension weights to each modality’s 64-dimensional projection, producing a fused 128-dimensional latent vector. This architecture inherits the global sequential sensitivity of the CNN and the local topological precision of the GNN, achieving the highest toxicity filter-pass rate (18.2%) and the highest QED targeted-generation hit rate (7.6%) among the three encoders.

5.1.3 Anti-Posterior-Collapse Training

Posterior collapse — where the encoder degenerates to near-prior distributions and the decoder completely disregards the latent vector — is the dominant failure mode of sequence VAEs. Seven complementary interventions were applied to prevent this: cyclical KL annealing (4 cycles over 25 epochs), free-bits regularisation enforcing a minimum KL contribution per latent dimension, word dropout that substitutes decoder input tokens with an unknown token at training time, persistent latent injection at every GRU decoding step, a deliberately weakened single-layer GRU decoder, zero-initialised log-variance weights, and stronger reparameterisation noise early in training. The combination of these techniques maintained active-unit counts of 37 (CNN) and 70 (GNN) under full $\beta = 1.0$ KL regularisation, confirming that collapse was avoided in both encoder modalities.

5.1.4 Property Prediction and Targeted Generation

A multi-output MLP was trained to predict three pharmaceutical properties — Quantitative Estimate of Drug-likeness (QED), the Wildman–Crippen partition coefficient (LogP), and the Synthetic Accessibility Score (SAS) — directly from 128-dimensional latent vectors. All three outputs converged under MSE loss, and prediction accuracy was sufficient to serve as a real-time gradient optimisation signal. Targeted generation was implemented as gradient descent on $\mathbf{z}$ against the property predictor objective toward user-specified QED, LogP, and SAS targets.

Across nine property settings and three encoders, single-property hit rates of 6.1–13.0% were achieved at medium tolerance thresholds, consistent with published results for VAE systems employing fixed-decoder latent optimisation.

5.1.5 Toxicity Filtering and Platform Integration

All generated molecules were passed through a four-stage structural alert pipeline — Lipinski Rule of Five, PAINS, Brenk, and NIH filters — implemented via RDKit’s `FilterCatalog`. The Hybrid encoder achieved an overall filter-pass rate of 18.2%, representing an 82% relative improvement over the CNN baseline of 10.0%. This improvement is attributed to the GNN branch’s explicit encoding of formal charge and bond order, which biases optimised latent trajectories away from configurations containing reactive charged heteroatom groups.

The SmartChem web platform — comprising a React frontend, a FastAPI backend, an asynchronous MongoDB-backed job queue, and a browser-side 3Dmol.js molecular viewer — integrates all components into a unified workflow. Synchronous API endpoints responded within 120 ms under all tested conditions, and an integrated Llama-3.3-70B AI chemistry assistant achieved a correct-response rate of 93% on natural-language property and toxicity queries. The platform demonstrates that molecular generation, interactive 3D visualisation, multi-stage toxicity filtering, and AI-assisted interpretation can be delivered together within a single, user-accessible system.

5.1.6 Summary of Results

The principal quantitative findings of the project are as follows:

- **100% structural validity** and **100% novelty** across all 6,750 generated molecules, with a mean pairwise diversity of 0.895–0.897 indicating the absence of mode collapse.
- **Strong latent utilisation:** 37 active units for the CNN encoder and 70 for the GNN encoder under full KL regularisation, confirming that anti-collapse interventions were effective.
- **Superior Hybrid encoder performance:** an 18.2% filter-pass rate and a 7.6% QED hit rate, outperforming the CNN (10.0%; 6.1%) and GNN (17.6%; 7.0%) baselines.
- A functional full-stack platform with sub-120 ms API response times, asynchronous generation job processing, and integrated AI assistance.

SmartChem demonstrates that it is possible to combine molecular generation, representation learning, property prediction, cheminformatics filtering, and a deployable web interface into a coherent and technically defensible system at student scale. The identified limitations — principally the predictor-decoder semantic gap and the fragment-size bias in the decoder — are well-understood architectural constraints with established remediation strategies, and they define a clear roadmap for further development.

5.2 Future Work

A number of directions for expanding and improving SmartChem are identified based on the limitations observed during experimentation.

5.2.1 Reinforcement Learning Fine-Tuning of the Decoder

The most significant near-term improvement is the application of reinforcement learning (RL) to fine-tune the GRU decoder directly against RDKit-computed reward signals for QED, LogP, and SAS. The existing predictor-decoder semantic gap — in which gradient optimisation in latent space moves $\mathbf{z}$ toward higher predictor-score regions, but the frozen decoder produces sequences with lower actual properties — is architecturally inherent to the fixed-decoder regime. An RL fine-tuning stage using policy gradient methods such as REINFORCE or Proximal Policy Optimisation would align the decoder’s generation distribution with true molecular property rewards and is expected to substantially improve single-property and joint hit rates. This is the closest analogue to the REINVENT system, which achieves demonstrably higher targeted hit rates through the same mechanism.

5.2.2 Correcting Fragment-Size Bias

The decoder exhibits a systematic fragment-size bias in which generated molecules have a mean molecular weight of approximately 35–50 Da, well below the 300–500 Da range typical of oral drug candidates. This occurs because the GRU decoder assigns high probability to early sequence termination, producing short SELFIES strings that correspond to small fragments. Future work should introduce a length-conditioned reward signal during training, penalise early termination, or augment the training distribution with molecules sampled uniformly across the full drug-like molecular weight range. Correcting this bias is a prerequisite for meaningful QED and SAS optimisation, as both metrics penalise small fragments independently of the optimisation direction.

5.2.3 Integration of Molecular Docking and Protein Target Information

The current system performs target-agnostic property optimisation with no information about specific protein binding sites. A natural and clinically significant extension is to include molecular docking scores — such as those from AutoDock

Vina or GNINA — as additional reward signals for the RL fine-tuning stage. Conditioning generation on a protein target’s binding pocket descriptors, or on the sequence of a known therapeutic target, would enable genuinely structure-based *de novo* design, bridging the gap between the current physicochemical property optimisation and the actual task of designing selective protein ligands.

5.2.4 Three-Dimensional Geometric Molecular Representations

The current encoder architectures — CNN on SELFIES sequences and GNN on 2D molecular graphs — do not encode three-dimensional conformational geometry. Incorporating 3D geometric deep learning through equivariant graph neural networks (e.g., SE(3)-Transformer, EGNN, or DimeNet) would allow the encoder to represent bond angles, dihedral torsions, and spatial atom arrangements, which are critical for shape-complementarity to binding pockets and for predicting ADMET properties sensitive to molecular geometry. This extension would also enable the direct generation of 3D conformers rather than requiring a separate conformer embedding step.

5.2.5 Training on Larger Datasets and for More Epochs

All models were trained on a 100,000-molecule filtered subset of ZINC-250k, constrained by single-GPU compute availability. Training on the full ZINC-250k dataset (approximately 250,000 molecules) or on larger curated collections such as ChEMBL or ZINC15 (millions of drug-like molecules) would expose the encoders to a substantially broader region of chemical space. Increased training duration would also be expected to improve latent utilisation and reconstruction quality for both CNN and GNN models, reducing the reconstruction loss floor and producing more robust property predictor training data.

5.2.6 Extended Property and Toxicity Endpoint Coverage

The current property predictor covers three endpoints (QED, LogP, SAS). Future versions should extend this to additional ADMET (Absorption, Distribution, Metabolism, Excretion, and Toxicity) properties relevant to drug development, including aqueous solubility (logS), Caco-2 membrane permeability, human intestinal absorption, plasma protein binding, hERG channel inhibition risk, and metabolic stability. On the toxicity filtering side, structural alert rules beyond the current four-stage pipeline — such as REOS filters, skin sensitiser alerts, or mutagenicity indicators from Ames test QSAR models — would provide more

comprehensive preclinical screening. A dedicated multi-task ADMET predictor trained on ChEMBL bioassay data would represent a significant improvement over the current three-output MLP.

5.2.7 Wet-Laboratory Validation

All outputs from SmartChem are computational candidates that have not been experimentally confirmed. Future work should include wet-laboratory validation of a selected subset of filter-passing generated molecules through chemical synthesis and biological assay. Priority candidates would be identified by combining high QED, favourable LogP, low SAS, and — once docking is integrated — strong predicted binding affinity to a specific therapeutic target. Experimental validation would provide ground-truth feedback on the accuracy of the computational screening pipeline and is a necessary step toward any genuine drug discovery application.

5.2.8 Cloud Deployment and Scalability

The current platform runs on a single local machine, limiting concurrent generation capacity to a small number of simultaneous jobs. Deployment on cloud infrastructure — for example, containerised services on AWS, Google Cloud, or Azure, with GPU-backed generation workers managed by a job scheduler such as Celery — would allow the platform to serve multiple users in parallel and support generation runs at scales of thousands of molecules per request. A cloud deployment would also enable model version management, continuous integration of updated encoder weights, and access control suitable for academic or early-stage industrial use.

5.3 Closing Remarks

SmartChem represents a fully operational implementation of a molecular VAE-based drug discovery platform, spanning representation design, model training, property prediction, targeted generation, toxicity filtering, and web deployment. The system has met its stated objectives and has identified, with precision, the architectural constraints that limit its current performance. The future directions outlined above are grounded in the experimental observations reported in Chapter ?? and follow established research trajectories in computational drug discovery. Addressing them incrementally would progressively close the gap between the current prototype and a system capable of supporting genuine early-phase drug discovery research.

Bibliography

- [1] Author Name, "Paper Title", Journal, Year.
- [2] Author Name, "Book Title", Publisher, Year.

Chapter A

Appendix A

Additional data or code.